Tutoriel sur RIOT OS et LoRa/LoRaWAN avec la carte Wyres Base

Par:
Ngbwa Massaga Darrel Cameron
Nkwenti Flora Pansy
Djifack Line Audrey
Djifo Taboh Manuela

Sous la supervision de: **Dr Chana Anne-Marie**

Contents

1 L'ÉCOSYSTÈME DE L'IOT **1**
- 1.1 Appareils IoT haut de gamme . . . 1
- 1.2 Appareils IoT bas de gamme . . . 1
- 1.3 LE SYSTEME D'EXPLOITATION RIOT-OS . . . 1
- 1.4 Structure d'une application RIOT . . . 2
- 1.5 Construction d'une application RIOT . . . 2
- 1.6 Exécution d'une application RIOT . . . 3

2 Installations Nécessaires **3**
- 2.1 Prérequis . . . 3
- 2.2 Installation de la chaîne de développement RIOT pour processeur ARM . . . 4
- 2.3 Installation de la chaîne de développement RIOT pour processeur ARM via Docker . . . 4
- 2.4 Clonage des dépots . . . 5
- 2.5 Installation des pilotes des adaptateurs USB Serial . . . 5
- 2.6 Consoles UART . . . 5
- 2.7 Environnement de développement . . . 6
- 2.8 Docker et Docker Compose . . . 6

3 La carte Wyres Base **6**
- 3.1 Caractéristiques techniques . . . 7
- 3.2 Connexion . . . 7
- 3.3 Branchements pour la programmation . . . 8
- 3.4 Branchement de FTDI pour le connecteur Grove de la console UART (shell) . . . 9
- 3.5 Branchement sur le connecteur Grove I2C . . . 9

4 Premiers pas avec RIOT et la carte Wyres **9**
- 4.1 Configuration de la carte cible pour `make` . . . 9
- 4.2 Connexion à la console . . . 10
- 4.3 Compilation du firmware . . . 10
- 4.4 Explication des résultats de compilation . . . 10
- 4.5 Analyse des fichiers produits . . . 11
- 4.6 Chargement du firmware . . . 11
- 4.7 Utilisation de la console (tio) . . . 11
- 4.8 Utilisation des Timers . . . 11
- 4.9 Threads . . . 11
- 4.10 RTC (Realtime Clock) . . . 12
- 4.11 WDT (Watchdog Timer) . . . 12
- 4.12 EEPROM . . . 13
- 4.13 Tension de l'alimentation . . . 13

5 Pilotage des entrées-sorties de la carte **13**
- 5.1 Configuration de la Carte . . . 13
- 5.2 LED / DEL . . . 13
- 5.3 Programme de Clignotement . . . 13
- 5.4 Bouton Poussoir . . . 14
  - 5.4.1 Programme de Suivi de l'État du Bouton . . . 14
  - 5.4.2 Bouton Poussoir en Mode Interruption . . . 14
- 5.5 Entrée/Sortie Externe . . . 15
- 5.6 Capteur Analogique de Luminosité . . . 15

6 Pilotage des capteurs MEMS **16**
6.1 Quelques exemples courants de capteurs MEMS : . . . 16
6.2 Configuration initiale . . . 16
6.2.1 Préparation de la carte cible pour `make` . . . 16
6.2.2 Connexion à la console avec `tio` . . . 16
6.3 Capteur de pression atmosphérique ST LPS22HB . . . 17
6.4 Accéléromètre LIS2DE12 . . . 17
6.5 Capteurs I2C externes . . . 17
6.6 BME280 . . . 18
6.7 BME680 . . . 18
6.8 MPL3115A2 . . . 19

7 Communications LoRa **20**
7.1 Rappel de quelques notions . . . 20
7.1.1 Configuration du développement . . . 20
7.1.2 Instructions pour Linux . . . 21
7.1.3 Instructions pour Mac OS . . . 21
7.1.4 Notes supplémentaires . . . 21
7.2 Les éléments les plus importants dans la communication LoRa . . . 21
7.2.1 La modulation de communication LoRa . . . 21
7.2.2 Communication point-à-point (P2P) avec LoRa . . . 21
7.2.3 Commandes pour le développement . . . 22
7.2.4 LoRaChat . . . 22
7.2.5 Observation depuis les stations . . . 22

8 Communications LoRa (avancé) **23**
8.1 L'infrastructure LoRaWAN . . . 23
8.2 Les éléments . . . 23
8.3 Les identifiants et les clés . . . 24
8.4 Classes d'objet dans LoRaWAN . . . 25

9 Collecte,stockage et visualisation des données des capteurs LORaWAN **25**
9.1 Nodered - Influxdb - Grafana (on premise) : Collecte, stockage et affichage des mesures de capteurs LoRaWAN . . . 25
9.1.1 Lancement de la composition . . . 25
9.1.2 Collecte et stockage . . . 25
9.1.3 Visualisation . . . 26
9.1.4 Ajout d'un panel . . . 26
9.1.5 Envoi de messages vers les équipements . . . 26
9.1.6 Inspection de la base de données InfluxDB . . . 28
9.1.7 Export des séries temporelles depuis la base de données InfluxDB . . . 28
9.1.8 Import d'une série temporelle dans la base de données InfluxDB . . . 28
9.1.9 Qu'est-ce qu'InfluxDB ? . . . 29
9.1.10 Quand utiliser InfluxDB ? . . . 29
9.1.11 Sauvegarde/Restauration d'une base de données InfluxDB . . . 30
9.2 Thingsboard (cloud) : Collecte, stockage et affichage des mesures de capteurs LoRaWAN. . . . 30
9.2.1 Création d'un compte Thingsboard . . . 30
9.2.2 Configuration de l'équipement . . . 32
9.2.3 Configuration de l'intégration dans le LNS . . . 34
9.3 Thingsboard (on premise) : Collecte, stockage et affichage des mesures de capteurs LoRaWAN . . . 34
9.4 Influx Data (cloud) : Collecte, stockage et affichage des mesures de capteurs LoRaWAN. . . . 35
9.5 Autres plateformes IoT : Collecte, stockage et affichage des mesures de capteurs LoRaWAN . . . 37
9.5.1 MyDevices . . . 37
9.5.2 Datacake . . . 38
9.5.3 Azure IoT . . . 38
9.5.4 Amazon IoT . . . 38
9.5.5 IBM IoT . . . 38

9.6 Collecte des mesures de capteurs LoRaWAN en ligne de commande: . . . 38
9.6.1 Installation de nvm . . . 39
9.6.2 Collecte . . . 39
9.6.3 Convertion NDJSON vers CSV . . . 39
9.7 Notebook Jupyter : Collecte, stockage et affichage des mesures de capteurs LoRaWAN : . . . 39
9.7.1 Explication du code . . . 40

10 Langages de programmation de haut niveau avec RIOT 40
10.1 C++ . . . 40
10.1.1 Utilisation d'une bibliothèque C++ depuis un programme C . . . 41
10.2 MicroPython . . . 41
10.3 Rust . . . 42
10.3.1 Fonctionnement . . . 42
10.3.2 Les composants de bibliothèque en Rust . . . 42
10.3.3 Toolchain . . . 42
10.3.4 Installation . . . 43
10.4 WASM (Web Assembly) . . . 43
10.5 LUA . . . 43
10.6 Javascript (jerryscript) . . . 43

11 Cryptographie avec RIOT et la carte Wyres Base 44
11.1 Hachage . . . 44
11.2 Chiffrage et déchiffrage à clé symétrique (AES ...) . . . 44
11.3 Cryptographie sur les courbes elliptiques . . . 44
11.4 Tests de cryptographie sur les courbes elliptiques . . . 45

12 Intelligence artificielle embarquée (TinyML) 45
12.1 TinyML . . . 45
12.1.1 Les principales caractéristiques du TinyML . . . 45
12.1.2 Quelques cas d'utilisation du TinyML . . . 45
12.1.3 Présentation des principaux défis du TinyML . . . 46
12.2 Travail Pratique . . . 46
12.2.1 Configurez la carte cible pour make . . . 46
12.2.2 Qu'est-ce que RIOT ? . . . 46
12.3 TensorFlow Lite Micro . . . 46
12.4 Frameworks d'IA embarqués . . . 46
12.5 Intégration de TinyML dans RIOT . . . 46
12.5.1 Optimisation des modèles d'IA . . . 46
12.5.2 Intégration dans l'application . . . 46
12.5.3 Exemples et cas d'usage . . . 47
12.5.4 Outils de développement . . . 47
12.5.5 Écosystème et communauté . . . 47
12.6 TensorFlow Lite for Microcontrollers (TensorFlow Lite Micro) . . . 47
12.6.1 Applications typiques de TensorFlow Lite Micro sur RIOT . . . 47
12.6.2 Intégration de TensorFlow Lite Micro dans RIOT . . . 47

13 Gestion d'énergie avec RIOT et la carte Wyres Base 48
13.1 Mesure de la consommation électrique . . . 48
13.1.1 Nordic PPK II . . . 48
13.1.2 ST X-NUCLEO-LPM01A Power Shield . . . 50
13.2 Principes de programmation basse consommation d'énergie . . . 51
13.2.1 Gestion de l'énergie dans RIOT . . . 51

List of Figures

1 MakeFile . . . 2
2 le fichier C . . . 2
3 La carte Wyres Base . . . 7
4 Programmateur ST-Link . . . 8
5 Carte Wyres Base avec connecteur Grove I2C . . . 10
6 Pseudo-Bouton avec les fils sur la carte wyres Base . . . 14
7 Platine I2C Hub . . . 18
8 BME280 . . . 18
9 BME680 . . . 19
10 MPL3115A2 . . . 20
11 Console tio . . . 22
12 Topologie des reseau LoRaWAN . . . 23
13 Illustration du cas de panne d'un équipement . . . 24
14 Les équipements d'un réseau LoRaWAN . . . 24
15 Interface NodeRed 1 . . . 26
16 Interface NodeRed 2 . . . 26
17 Interface Manipulation carte Wyres Base . . . 27
18 Interface manipulation carte Wyres Base 2 . . . 27
19 Presentation interface ajout panel . . . 27
20 Presentation schema des messages echangees . . . 27
21 Presentation du message Downlink . . . 28
22 FabMSTIC 20cac . . . 28
23 Interface ajout device type Wyres Base 1 . . . 31
24 Interface ajout device type Wyres Base 2 . . . 31
25 Interface Thingboard . . . 32
26 Interface presentation activite device Wyres Base 12345678 . . . 32
27 Interface creation device . . . 32
28 Interface configuration device Wyres Base Riot 12345678 . . . 33
29 Interface ThingBoard ajout Widget . . . 33
30 Interface ThingBoard; presentation des Root Rule Chain . . . 33
31 Interface ThingBoard CalpusloT . . . 35
32 Interface Influx Data Cloud 1 . . . 35
33 Interface Influx Data Cloud generation token . . . 36
34 Interface InfluxData cloud Data Explorer . . . 37
35 Interface Influx Data cloud Mon premier tableau de bord IoT . . . 37
36 Interface serveur chipstark intergration influxDB . . . 38
37 Resultat de la compilation d'un code C++ avec RIOT . . . 41
38 Resultat de la compilation des threads C++ avec RIOT . . . 41
39 Le PPK2 . . . 49
40 Informations sur la consommation d'énergie en temps réel . . . 50
41 Le X-NUCLEO-LPM01A . . . 51
42 Programme pour tester les différents modes d'alimentation . . . 52
43 La suite du programme . . . 52
44 Observation de l'impact des modes d'alimentation en termes de consommation d'énergie. . . . 53
45 Le mode d'alimentation 0 . . . 54

Introduction

RIOT OS est un système d'exploitation open source conçu pour les appareils IoT et les systèmes embarqués. Il est particulièrement adapté aux microcontrôleurs à faible puissance. RIOT OS prend en charge une grande variété de matériels, y compris la carte Wyres Base qui intègre un module LoRa. LoRa est une technologie de modulation radio basse puissance qui permet des communications longue portée. LoRaWAN est un protocole réseau construit sur la couche physique LoRa. Il définit la couche MAC et la gestion du réseau. La carte Wyres Base est équipée d'un module LoRa qui permet de l'utiliser dans des applications LoRaWAN. RIOT OS fournit des pilotes et une pile réseau LoRaWAN qui permettent de tirer parti de ces capacités LoRa sur la Wyres Base. Les développeurs peuvent ainsi utiliser RIOT OS et les fonctionnalités LoRa/LoRaWAN de la Wyres Base pour créer des applications IoT basse consommation communicant sur de longues distances. Cela en fait une plateforme intéressante pour des projets tels que des réseaux de capteurs sans fil, des trackers, etc. L'objectif de ce tutoriel est la prise en main de la carte Wyres Base avec le système d'exploitation RIOT OS, effectuer des mesures avec les capteurs de la carte et des capteurs additionnels, transmettre des mesures de capteurs via LoRa et LoRaWAN, collecter, stocker et archiver les mesures des capteurs. Pour atteindre cet objectif, notre tutoriel sera structuré de la manière suivante. Nous commencerons par explorer le pilotage des entrées-sorties de la carte et des capteurs MEMS. Ensuite, nous nous plongerons dans les détails des communications LoRa et LoRaWAN, en abordant également des notions avancées de ces technologies. Nous verrons comment collecter, stocker et afficher les mesures des capteurs LoRa, en utilisant des langages de programmation de haut niveau pour faciliter le développement. Nous aborderons également des sujets avancés tels que la cryptographie et l'intelligence artificielle embarquée (TinyML), en mettant l'accent sur la programmation ultra basse consommation. Enfin, nous discuterons des contributions potentielles à l'écosystème RIOT OS et des opportunités de développement pour enrichir les capacités de la Wyres Base dans le domaine de l'IoT.

1 L'ÉCOSYSTÈME DE L'IOT

Dans le domaine de l'IoT, il existe des appareils IoT haut de gamme, des appareils IoT bas de gamme et Internet[1].

1.1 Appareils IoT haut de gamme

Les appareils IoT haut de gamme ont des processeurs puissants pour des tâches complexes comme l'analyse de données en temps réel, l'apprentissage automatique et l'informatique de périphérie[1]. Ils ont plus de mémoire (RAM).

1.2 Appareils IoT bas de gamme

Les appareils IoT bas de gamme ont des processeurs moins puissants, souvent optimisés pour une faible consommation. Ils ont des fonctionnalités plus simples, se concentrant sur la collecte et la transmission de données de capteurs[1]. RIOT est conçu pour ce type d'appareils bas de gamme.

1.3 LE SYSTEME D'EXPLOITATION RIOT-OS

RIOT-OS est un système d'exploitation destiné aux appareils IoT (Internet des Objets) à ressources limitées, c'est-à-dire aux appareils équipés de microcontrôleurs[1].
Ses principales caractéristiques sont les suivantes :

- Il a une architecture de micro-noyau, ce qui signifie qu'il nécessite très peu de ressources (RAM = 2,8 Ko, ROM = 3,2 Ko).
- Il est en temps réel et multithread.
- Il est doté de piles de réseautage développées en interne (implémentation de protocoles réseau sur mesure).

RIOT est une plateforme et un écosystème open source.

- Il est gratuit et facile à utiliser.
- Son langage de programmation standard est le C.

- Son API est indépendante du matériel, c'est-à-dire que l'API elle-même ne se soucie pas du type spécifique de matériel avec lequel elle interagit.

1.4 Structure d'une application RIOT

Une application RIOT se compose de :

1. Un Makefile

```
APPLICATION = example

BOARD ?= native

RIOTBASE ?= $(CURDIR)/../../../RIOT

DEVELHELP ?= 1

include $(RIOTBASE)/Makefile.include
```

Figure 1: MakeFile

"APPLICATION = example" définit le nom de l'application à construire, ici "example".

"BOARD ?= native" définit la carte cible sur laquelle l'application sera exécutée. Le ?= signifie que la valeur sera utilisée si elle n'a pas déjà été définie ailleurs. Dans ce cas, la carte par défaut est "native", ce qui signifie que l'application sera exécutée sur la machine hôte (ordinateur de développement) et non sur un matériel embarqué.

"RIOTBASE ?= $(CURDIR)/../../../RIOT" définit le chemin vers la racine du projet RIOT, qui contient tous les fichiers sources et les outils de compilation.

"DEVELHELP ?= 1" active l'option de "développement" dans la compilation, ce qui inclut des fonctionnalités supplémentaires pour faciliter le débogage.

"include $(RIOTBASE)/Makefile.include" inclut le fichier Makefile principal de RIOT, qui contient toutes les règles et les variables nécessaires pour construire une application RIOT.

2. Un fichier C contenant la fonction principale (Main)

   La figure suivante nous présente le fichier C contenant la fonction Main

```c
#include <stdio.h>

int main(void)
{
    puts("My first RIOT application");
    return 0;
}
```

Figure 2: le fichier C

1.5 Construction d'une application RIOT

Le système de construction de RIOT est basé sur l'outil de construction make. Pour construire une application, `make` peut être appelé de 2 façons :

1. Depuis le répertoire de l'application :

```
$ cd <répertoire_application>
$ make
```

2. Depuis n'importe où, en utilisant l'option `-C` pour spécifier le répertoire de l'application :

```
$ make -C <répertoire_application>
```

1.6 Exécution d'une application RIOT

L'exécution de l'application dépend de la carte cible :

- Exécution sur un système natif : l'application RIOT est exécutée comme un simple processus Linux

```
$ make BOARD=native -C <application_dir>
$ <application_dir>/bin/native/application.elf
```

- Exécution sur du matériel : l'application RIOT doit d'abord être flashée sur la carte. Nous pouvons utiliser "flash" et "term" avec "make". "flash" est utilisé pour construire et écrire le micrologiciel dans la mémoire flash du MCU. "term" ouvre un client de terminal connecté au port série de la cible.

```
$ make BOARD=<target> -C <application_dir> flash term
```

2 Installations Nécessaires

2.1 Prérequis

Il faut avoir un ordinateur avec un système d'exploitation linux, ou MacOS, ou Windows, installé. Le mieux est d'avoir une version très récente pour avoir les dernières mises à jour et une version compatible avec RIOT OS. Les versions compatibles avec RIOT OS sont:

- Pour Linux:

```
Ubuntu:
    Ubuntu 20.04 LTS (Focal Fossa)
    Ubuntu 22.04 LTS (Jammy Jellyfish)
Debian:
    Debian 10 (Buster)
    Debian 11 (Bullseye)
Fedora:
    Fedora 33
    Fedora 34
    Fedora 35
CentOS/RHEL:
    CentOS 7
    CentOS 8
    Red Hat Enterprise Linux (RHEL) 7
    Red Hat Enterprise Linux (RHEL) 8
```

- Pour MacOS:

```
macOS 10.13 (High Sierra)
macOS 10.14 (Mojave)
macOS 10.15 (Catalina)
macOS 11 (Big Sur)
macOS 12 (Monterey)
```

- Pour Windows:

```
Windows 10 version 1903 (19H1) ou ultérieure
Windows 10 version 21H1 ou ultérieure
Windows 11 version 21H2 ou ultérieure
```

Il faut installer tous les logiciels nécéssaires à la programmation de la carte Wyres Base avec RIOT OS. Il faut avoir une machine avec le système d'exploitation linux ou MacOS de préférence une version récente.

2.2 Installation de la chaîne de développement RIOT pour processeur ARM

Pour ce tutoriel nous nous attarderons sur les systèmes Linux et MacOS.

- **Sur Linux :** Il faut ouvrir le terminal et taper la commande suivante: ***sudo apt install git gcc-arm-none-eabi make gcc-multilib libstdc++-arm-none-eabi-newlib openocd gdb-multiarch doxygen wget unzip python3-serial***[2]. Cette commande installe les dernières versions des paquets suivants :
  - **git** : pour cloner les dépôts Git.
  - **gcc-arm-none-eabi** : le compilateur GCC pour les microcontrôleurs ARM.
  - **make** : pour la construction des projets.
  - **gcc-multilib** : bibliothèques 32 bits pour la compilation croisée.
  - **libstdc++-arm-none-eabi-newlib** : bibliothèque C++ pour les microcontrôleurs ARM.
  - **openocd** : pour le débogage et la programmation des microcontrôleurs.
  - **gdb-multiarch** : le débogueur GDB multiplateforme.
  - **doxygen** : pour la génération de documentation.
  - **wget** : pour télécharger des fichiers.
  - **unzip** : pour extraire des archives ZIP.
  - **python3-serial** : bibliothèque Python pour la communication série.

  Une fois ces outils installés, on peut cloner le dépôt RIOT OS, configurer l'environnement de développement et commencer à travailler sur des applications IoT utilisant la carte Wyres Base.

- Sur MacOS: Il faut ouvrir le terminal et taper la commande suivante: ***brew install gcc-arm-embedded open-ocd make***[2]

2.3 Installation de la chaîne de développement RIOT pour processeur ARM via Docker

La toolchain de RIOT peut aussi etre installée via un conteneur Dockeur pour ceux qui sont à l'aise avec Dockeur [2]. Pour le faire il faut ouvrir le terminal et taper les commandes suivantes:

```
export BUILD_IN_DOCKER=1

cd ~/github/RIOT-OS/RIOT/examples/hello-world/
export BOARD=wyres-base
export EXTERNAL_BOARD_DIRS=~/github/campusiot/RIOT-wyres/boards
make -j 4
```

2.4 Clonage des dépots

Maintenant que l'environnement de développement est installé il faut cloner les différents dépots github de CampusIot, RIOT OS, thingsat. Dans le terminal, il faut taper les commandes:

mkdir -p /github/RIOT-OS
git clone git@github.com:RIOT-OS/RIOT.git /github/RIOT-OS/RIOT
mkdir -p /github/campusiot
git clone git@github.com:CampusIoT/RIOT-wyres.git /github/campusiot/RIOT-wyres
mkdir -p /github/thingsat
git clone git@github.com:thingsat/ecc_mcu_benchmarks.git /github/thingsat/ecc_mcu_benchmarks
git clone git@github.com:thingsat/riot_modules.git /github/thingsat/riot_modules

Ces commandes vont créer trois dossiers RIOT-OS, campusiot, thingsat et cloner les différents dépots dans ces dossiers respectivement. Ensuite il faut vérifier les programmateurs utilisables en tapant les commandes:

```
cd ~/github/RIOT-OS/RIOT/examples/hello-world/
export BOARD=wyres-base
export EXTERNAL_BOARD_DIRS=~/github/campusiot/RIOT-wyres/boards
make info-programmers-supported
```

2.5 Installation des pilotes des adaptateurs USB Serial

Il faut installer les pilotes des adappteurs USB Serial [2]pour que lorsqu'on branche son matériel sur la machine, que la machine reconnaisse le matériel etque la connexion USB permettent le transfert de données. Les pilotes les plus importants sont FT234R, CH340, CP210X [2]. Voici les étapes pour installer les pilotes FT234R, CH340 et CP210X sur Linux :

- **FT234R** :
  - Sur les distributions basées sur Debian (Ubuntu, Mint, etc.) : Installez le package 'libftdi-dev' via la commande 'sudo apt-get install libftdi-dev .[2]
  - Sur les distributions basées sur Fedora (CentOS, RHEL, etc.) : Installez le package 'libftdi-devel' via la commande 'sudo dnf install libftdi-devel .[2]
- **CH340** :
  - Télécharger le pilote CH340 depuis le site officiel : https://github.com/felHR85/UsbSerial/blob/master/CH341SER.zip
  - Décompresser l'archive et accéder au dossier contenant les fichiers du pilote.
  - Compiler et installer le pilote avec les commandes suivantes : - ***sudo make*** - ***sudo make install*** [2]
- **CP210X** :
  - Sur les distributions basées sur Debian (Ubuntu, Mint, etc.) : Installer le package **silabs-bus-enumerator** [2] via la commande: ***sudo apt-get install silabs-bus-enumerator***.
  - Sur les distributions basées sur Fedora (CentOS, RHEL, etc.) : Installer le package silabs-usbxpress-devel via la commande: ***sudo dnf install silabs-usbxpress-devel***. [2]

  Une fois les pilotes installés, on doit pouvoir utiliser les périphériques FT234R, CH340 et CP210X sur votre système Linux. Il faut également s'assurer de brancher les périphériques et de vérifier leur reconnaissance par le système avec la commande ***lsusb***.

2.6 Consoles UART

tio est un utilitaire très pratique pour échanger avec une console UART qui est une alternative à ***make term*** qui utilise ***pyterm***[2].

Pour l'installer selon [2], il faut taper les commandes suivantes:

- Sur Linux:

```
sudo apt install tio pour installer le paquet
tio --version
```

- Sur MacOS:

```
brew install tio
tio --version
```

2.7 Environnement de développement

Pour le développement sur STM32, il est recommandé d'utiliser STM32CubeIDE [2] mais Visual Studio Code est une alternative plus facile et plus rapide à prendre en main.

2.8 Docker et Docker Compose

Docker et Docker Compose seront utiles pour la partie sur la Collecte, le stockage et l'affichage des mesures de capteurs LoRaWAN ou pour l'exécution de la chaîne de développement avec Docker [2]. Il faut donc déjà précharger les images des conteneurs Docker qui seront utilisés lors de cette partie avec les commandes suivantes:

```
cd ~/github/campusiot/RIOT-wyres/backend
docker pull nodered/node-red
docker-compose pull
```

3 La carte Wyres Base

La carte Wyres Base est une carte électronique conçue par le projet Campus IoT. Elle dispose d'un microcontrôleur basse consommation STM32L151CC de ST Microelectronics [3], d'un module de communication LoRa SX1272 de Semtech, ainsi que de plusieurs capteurs I2C tels qu'un accéléromètre, un baromètre et un microphone.



Figure 3: La carte Wyres Base

3.1 Caractéristiques techniques

- Microcontrôleur STM32L151CC de ST Microelectronics
  - 256 kB de mémoire Flash, 32 kB de RAM, 8 kB d'EEPROM
  - 1 x UART, 2 x SPI, 1 x I2C
- Module de communication LoRa SX1272 de Semtech
- Capteurs internes:
  - Accéléromètre ST LIS2DE12 (I2C)
  - Baromètre ST LPS22HB (I2C)
  - Microphone MEMS SPH0641LM4H-1 (I2S)
  - Capteur de luminosité sur GPIO
- 2 LEDs
- 1 bouton poussoir
- 1 sortie PWM (transistor MOSFET)

3.2 Connexion

Identifier le port série de la carte. Suivant le système d'exploitation, la procédure varie légèrement :

- **Sur Linux** : Lister les ports série disponibles avec la commande: dmesg — grep tty . Le port série de la carte Wyres Base sera généralement /dev/ttyUSB0 [3] .
- **Sur MacOS** : Lister les ports série disponibles avec la commande: ls /dev/tty.usb*. Le port série de la carte Wyres Base sera généralement /dev/tty.usbmodem* [3].
- **Sur Windows** : Ouvrir le Gestionnaire de périphériques, dans la section "Ports (COM et LPT)", on devrait voir apparaître un nouveau port série lié à la carte Wyres Base, par exemple "USB Serial Device (COM3)".

Vérifier la connexion en ouvrant un terminal série. Il faut ouvrir un terminal et se connecter au port série de la carte :

- Sur Linux: sudo screen /dev/ttyUSB0 115200.
- Sur MacOS: sudo screen /dev/tty.usbmodem* 115200.
- Sur Windows, remplacer /dev/ttyUSB0 par le numéro de port COM correspondant.

On devrait voir apparaître des informations sur la carte Wyres Base dans le terminal. Si c'est le cas, la connexion est établie avec succès.

3.3 Branchements pour la programmation

Pour programmer la carte Wyres Base, on a besoin d'un programmateur ST-Link [3]. On peut utiliser un ST-Link v2, ST-Link v3 ou le programmateur intégré dans les cartes Discovery ou Nucleo.



Figure 4: Programmateur ST-Link

Le tableau qui suit montre les correspondances des broches entre le programmateur ST-Link et la carte Wyres Base selon [3] :

ST-Link ARM JTAG	Couleur	Wyres Base
Pin 2 : VDD_TARGET	Rouge	Pin 3 : VDD
Pin 9 : SWCLK	Jaune	Pin 4 : CLK
Pin 4 : GND	Noir	Pin 2 : GND
Pin 7 : SWDIO	Bleu	Pin 5 : DIO
Pin 15 : NRST	Vert	Pin 1 : NRST

Table 1: Correspondance des broches entre le programmateur ST-Link et la carte Wyres Base

Remarque : Sur les cartes Nucleo, les broches SWD sont étiquetées directement sur la carte.

Le programmateur STLink disponible sur les cartes Discovery et Nucleo peuvent être utilisés pour charger le programme d'une autre carte.

ST-Link Nucleo	Couleur	Wyres Base
SWD Pin 1: VDDTARGET (VDD from application)	Red	Pin 3: VDD
SWD Pin 2: SWCLK (clock)	Yellow	Pin 4: CLK
SWD Pin 3: GND (ground)	Black	Pin 2: GND
SWD Pin 4: SWDIO (SWD data input/output)	Blue	Pin 5: DIO
SWD Pin 5: NRST (RESET of target STM32)	Green	Pin 1: NRST

Table 2: Correspondance des broches entre ST-Link Nucleo et Wyres Base

3.4 Branchement de FTDI pour le connecteur Grove de la console UART (shell)

Le connecteur Grove UART de la carte permet de brancher un adaptateur USB-Serial (aka FTDI) pour afficher des traces sur la console de la carte [3]. Il faut connecter l'adaptateur à la machine et ensuite taper les commandes suivantes [3]:

- Sur Linux:

```
lsusb
ls /dev/ttyUSB*
tio -L
tio -b 115200 -m INLCRNL /dev/ttyUSB0
```

- Sur MacOS:

```
lsub
ls /dev/tty.usb*
tio -L
tio -b 115200 -m INLCRNL /dev/tty.usbserial-XXXX
```

3.5 Branchement sur le connecteur Grove I2C

Le connecteur Grove I2C sert à connecter un ou plusieurs composants I2C comme ceux listés dans la partie **Pilotage des capteurs MEMS** [3].

4 Premiers pas avec RIOT et la carte Wyres

4.1 Configuration de la carte cible pour `make`

Pour configurer la carte Wyres, il faut exécuter les commandes suivantes dans le terminal selon [4] :

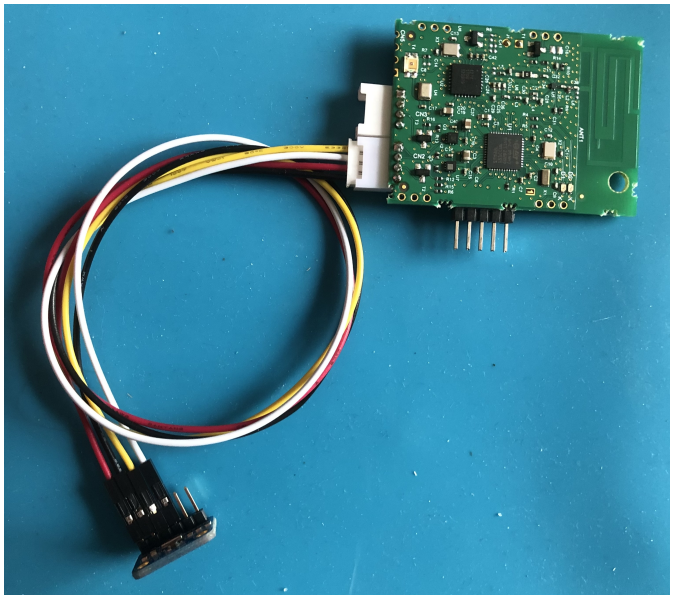


Figure 5: Carte Wyres Base avec connecteur Grove I2C

```
export BOARD=wyres-base
export EXTERNAL_BOARD_DIRS=~/github/campusiot/RIOT-wyres/boards
```

4.2 Connexion à la console

Rechercher le `tty` de la console et et il faut se connecter à celle-ci avec `tio` [4].

- Sur Linux

```
lsusb
tio
tio -b 115200 -m INLCRNL /dev/ttyUSB0
```

- Sur MacOS

```
lsusb
tio
tio -L
tio -b 115200 -m INLCRNL /dev/tty.usbserial-XXXX
```

4.3 Compilation du firmware

Pour construire le firmware, il faut utiliser la commande suivante :

```
cd ~/github/campusiot/RIOT-wyres/tests/shell
make -j 4
```

4.4 Explication des résultats de compilation

La commande `make` affiche les lignes suivantes d'après [4] :

```
text    data     bss     dec     hex filename
17124    132    1912   19168    4ae0 ~/github/campusiot/RIOT-wyres/tests/shell/bin/wyres-base/test-shel
```

Signification des différentes valeurs :

```
- \texttt{text} : le code et les constantes (et aussi la table de vecteurs).
- \texttt{data} : les variables initialisées. Cela compte à la fois pour la RAM et le FLASH.
- \texttt{bss} : les données non initialisées dans la RAM qui sont initialisées à zéro dans le code de
```

4.5 Analyse des fichiers produits

```
cat bin/$BOARD/riotbuild/riotbuild.h
ls -al bin/$BOARD
file bin/$BOARD/test-shell.*
```

Le fichier `.elf` est un fichier binaire contenant le firmware et des informations sur celui-ci [4]. Le fichier `.bin` est un fichier binaire contenant le firmware qui est écrit dans la mémoire Flash du STM32L151 à l'adresse 0x08000000 par `openocd`.

4.6 Chargement du firmware

Pour charger le binaire `.bin` dans la mémoire Flash (adresse 0x08000000) du STM32L151, utilisez la commande suivante d'après [4] :

make flash-only

Note: la commande `make -j 4 flash` enchaine les commandes `make -j 4` puis `make flash-only`.

4.7 Utilisation de la console (tio)

```
main(): This is RIOT! (Version: 2023.07-devel-325-g2863d)
test_shell.
> help
Command           Description
---------------------------------------
bufsize           Get the shell's buffer size
start_test        starts a test
end_test          ends a test
echo              prints the input command
empty             print nothing on command
app_metadata      Returns application metadata
pm                interact with layered PM subsystem
ps                Prints information about running threads.
reboot            Reboot the node
version           Prints current RIOT_VERSION
xfa_test1         xfa test command 1
xfa_test2         xfa test command 2
```

4.8 Utilisation des Timers

Les MCU disposent de plusieurs périphériques temporisateurs (timers). Les timers de RIOT simplifient leur usage pour déclencher une fonction après plusieurs micro-secondes ou milli-secondes [4].

```
cd ~/github/RIOT-OS/RIOT/tests/periph/timer_periodic/
make -j 4 flash
```

4.9 Threads

Les threads sont des processus légers partageant le même espace mémoire. Il est important de bien dimensionner la taille de la pile d'une thread pour éviter des dépassements [4]. EN d'autres termes, c'est pour faire des taches asynchrones.

```
cd ~/github/campusiot/RIOT-wyres/tests/shell
make -j 4 flash
```

4.10 RTC (Realtime Clock)

La plupart des microcontroleurs possède un périphérique horloge temps-réel (en anglais Realtime Clock, RTC) qui maintient la date et l'heure courante et définir des alarmes tant que le microcontroleur est alimenté [4]. Il est possible de supporter la perte d'alimentation (par exemple, changement de batterie) en ajoutant une pile de secours à cette horloge.

```
cd ~/github/RIOT-OS/RIOT/tests/periph/rtc/
make -j 4 flash
```

Modifier le *Makefile* de l'exemple shell pour ajouter:

```
FEATURES_REQUIRED += periph_rtc
FEATURES_OPTIONAL += periph_rtc_ms
FEATURES_OPTIONAL += periph_rtc_mem

DISABLE_MODULE += periph_init_rtc
```

Reflasher la carte avec les commandes suivantes:

```
cd ~/github/campusiot/RIOT-wyres/tests/shell
make -j 4 flash
```

4.11 WDT (Watchdog Timer)

Le chien de garde (Watchdog Timer en anglais) est un périphérique qui veille à redémarrer (reboot) le microcontroleur quand le programme a planté (débordement de pile) ou boucle [4].

```
cd ~/github/RIOT-OS/RIOT/tests/periph/wdt
make -j 4 flash
```

setup 0 10000 fixe le WDG à 10 seconds. Il faut ensuite lancer le watchdog avec la commande ***start***. Sans *kick*, il reboote la carte. Dans la console tio on a:

```
This is RIOT! (Version: 2023.07-devel-325-g2863d)
RIOT wdt test application
> help
Command              Description
---------------------------------------
range                Return wdt Timer range
setup                Setup wdt Timer
kick                 Delay wdt timer
start                Start wdt timer
startloop            Start wdt timer, loop print system time
> setup
usage: setup
> setup 0 10000
valid configuration
> start
starting wdt timer
> kick
delaying wdt timer
> kick
delaying wdt timer
> main(): This is RIOT! (Version: 2023.07-devel-325-g2863d)
RIOT wdt test application
```

4.12 EEPROM

L'EEPROM est une (petite) mémoire persistante qui est en général utilisé des paramêtres de personnalisation ou de configuration de l'équipement [4]. Le microcontroleur de la carte Wyres Base a une EEPROM de 8096 bytes.

```
cd ~/github/RIOT-OS/RIOT/tests/periph/eeprom
make -j 4 flash
```

Attention: il faut mieux ne pas stocker des secrets (clés cryptographiques par exemple) dans l'EEPROM. Il est préférable d'utiliser la mémoire persistante sécurisée d'un Secure Element (SE) intégré au microcontroleur ou d'un composant externe. Le MCU de la carte Wyres n'est pas doté d'un tel composant SE.

4.13 Tension de l'alimentation

Connaitre la tension d'alimentation de son matériel permet de savoir à quel moment une opération de maintenance doit être planifiée pour remplacer la batterie.

```
cd ~/github/RIOT-OS/RIOT/tests/periph/vbat
make -j 4 flash
```

5 Pilotage des entrées-sorties de la carte

Dans cette, on apprend à utiliser les GPIO numériques et analogiques en entrée et en sortie.

5.1 Configuration de la Carte

Il faut configurer la carte cible pour `make`:

```
export BOARD=wyres-base
export EXTERNAL_BOARD_DIRS=~/github/campusiot/RIOT-wyres/boards
```

Ensuite il faut rechercher le `tty` de la console et se connecter à celle-ci avec `tio`.

- Sur Linux

  ```
  lsusb
  tio
  tio -b 115200 -m INLCRNL /dev/ttyUSB0
  ```

- Sur MacOS

  ```
  lsusb
  tio
  tio -L
  tio -b 115200 -m INLCRNL /dev/tty.usbserial-XXXX
  ```

5.2 LED / DEL

La carte Wyres Base possède deux LED/DEL: une rouge (GPIO PA15) et une verte (GPIO PA0) [5]. Les broches des DEL/LED sont définies dans le fichier `board.h` de la carte.

5.3 Programme de Clignotement

Le programme ci-dessous fait clignoter les LED:

```
cd ~/github/campusiot/RIOT-wyres/tests/leds
make -j 4 flash
```

Console (`tio`)

```
main(): This is RIOT! (Version: 2023.07-devel-325-g2863d)
Toggle on board LEDs
```

5.4 Bouton Poussoir

ATTENTION! Toutes les cartes Wyres Base ne sont pas équipées d'un bouton. Pour émuler un bouton [5], on peut brancher deux fils sur les broches comme sur la photo ci-dessous et les faire toucher.

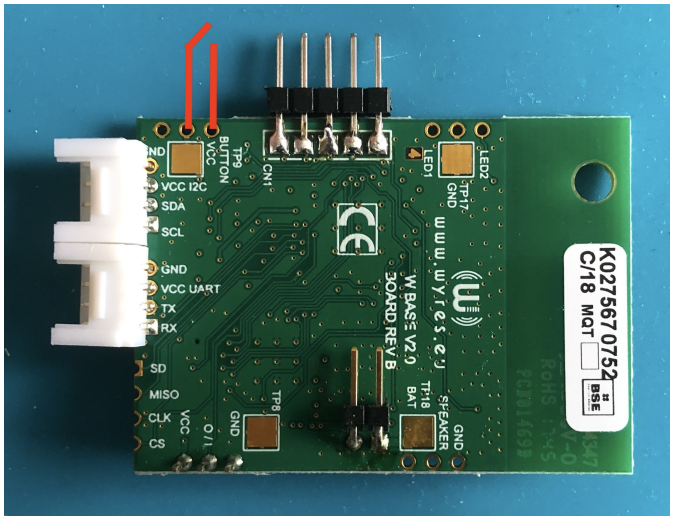


Figure 6: Pseudo-Bouton avec les fils sur la carte wyres Base

5.4.1 Programme de Suivi de l'État du Bouton

Pour les cartes équipées, le programme suivant suit l'évolution de l'état du bouton qui est sur la GPIO PB3 du STM32 [5]:

```
cd ~/github/campusiot/RIOT-wyres/tests/button
make -j 4 flash
```

Console (`tio`)

```
main(): This is RIOT! (Version: 2024.01-devel-168-g14ff95)
Read Button 0 state
Button 0 has been pressed!
Button 0 has been released!
```

Note: Pour éviter le risque de rebond (rebounce), il faut soit traiter logiciellement (avec une non-prise en compte des interruptions pendant une milliseconde), soit ajouter un condensateur dans le montage.

5.4.2 Bouton Poussoir en Mode Interruption

L'utilisation des interruptions sur les broches en entrée évite la scrutation active et permet d'économiser la batterie en mettant la MCU en sommeil profond [5].

```
cd ~/github/RIOT-OS/RIOT/tests/periph/gpio
make -j 4 flash
```

Console (`tio`)

```
> init_int
usage: init_int    [pull_config]
    flank:
    0: falling
    1: rising
    2: both
    pull_config:
```

```
    0: no pull resistor (default)
    1: pull up
    2: pull down
> init_int 1 3 0
GPIO_PIN(1, 3) successfully initialized as ext int
> enable_int 1 3 1
enabling GPIO interrupt
> INT: external interrupt from pin 3
```

5.5 Entrée/Sortie Externe

La carte Wyres Base possède une GPIO externe PA8 à usage général (entrée numérique ou sortie numérique) qui est sur le pad TP3 [5].

```
cd ~/github/RIOT-OS/RIOT/tests/periph/gpio
make -j 4 flash
```

Console (`tio`)

```
> init_in_pu 0 8
> read 0 8
GPIO_PIN(0.08) is HIGH
> read 0 8
GPIO_PIN(0.08) is LOW
```

Relier un fil Dupont entre GND et I/O pour constater le passage à l'état bas sur la GPIO PA8.

Certaines cartes sont munies d'une balance magnétique (switch) : placer un aimant dessus et relever la valeur de la GPIO. Elle devient LOW.

Pour tester, le reboot magnétique avec un aimant, il faut replacer la fonction suivante dans le code de l'exemple:

```
#ifdef MODULE_PERIPH_GPIO_IRQ
#include "periph/pm.h"
static void cb(void *arg)
{
    printf("INT: external interrupt from pin %i\n", (int)arg);
    pm_reboot();
}
#endif
```

5.6 Capteur Analogique de Luminosité

La carte Wyres Base possède un capteur de luminosité raccordé en entrée à la broche PA3 analogique du STM32 [5]. La broche du capteur de luminosité est définie dans le fichier `board.h` de la carte.

Programme de Test du Capteur de Luminosité[5] Le programme ci-dessous fait clignoter le capteur. Il faut relir le fichier `main.c`.

```
cd ~/github/campusiot/RIOT-wyres/tests/light-sensor
make -j 4 flash
```

On peut utiliser la lumière de la torche du téléphone pour éblouir le capteur pendant l'exécution du programme pour faire varier la mesure du capteur.

Console (`tio`)

```
main(): This is RIOT! (Version: 2023.07-devel-325-g2863d)
Test light sensor
Successfully initialized ADC_LINE(0)
ADC_LINE(0): raw value: 0016, percent: 00 %
ADC_LINE(0): raw value: 0027, percent: 00 %
ADC_LINE(0): raw value: 0304, percent: 07 %
```

6 Pilotage des capteurs MEMS

Les capteurs MEMS, ou Micro-Electro-Mechanical Systems (Systèmes Micro-Électro-Mécaniques), sont des dispositifs miniaturisés qui intègrent des éléments mécaniques et électriques sur une seule puce de silicium [6]. Ces capteurs sont largement utilisés dans diverses applications en raison de leur petite taille, de leur faible coût et de leur capacité à mesurer des phénomènes physiques avec une grande précision.

6.1 Quelques exemples courants de capteurs MEMS :

- Accéléromètres : Mesurent l'accélération linéaire dans une ou plusieurs directions. Utilisés dans les smartphones, les véhicules, les dispositifs de fitness, etc [6].
- Gyroscopes : Mesurent la vitesse de rotation et sont souvent combinés avec des accéléromètres pour fournir des informations complètes sur le mouvement. Utilisés dans les drones, les jeux vidéo, les smartphones, etc.
- Capteurs de pression : Mesurent la pression des gaz ou des liquides. Utilisés dans les applications médicales, les appareils de mesure météorologique, les systèmes de surveillance de la pression des pneus, etc.
- Microphones : Convertissent les ondes sonores en signaux électriques. Utilisés dans les téléphones, les appareils auditifs, les systèmes d'assistance vocale, etc.
- Capteurs de température : Mesurent la température ambiante. Utilisés dans les appareils électroniques, les systèmes de climatisation, les équipements médicaux, etc.
- Capteurs de proximité : Détectent la présence d'objets à proximité sans contact physique. Utilisés dans les smartphones pour éteindre l'écran pendant les appels, les dispositifs de sécurité, etc [6].

Dans cette partie, on apprend à récolter les mesures de capteurs numériques (MEMS) de la carte Wyres Base ainsi que celles de capteurs connectés via le connecteur Grove I2C de la carte.

6.2 Configuration initiale

6.2.1 Préparation de la carte cible pour `make`

```
export BOARD=wyres-base
export EXTERNAL_BOARD_DIRS=~/github/campusiot/RIOT-wyres/boards
```

6.2.2 Connexion à la console avec `tio`

- Sur Linux :

```
lsusb
tio
tio -b 115200 -m INLCRNL /dev/ttyUSB0
```

- Sur MacOS :

```
lsusb
tio
tio -L
tio -b 115200 -m INLCRNL /dev/tty.usbserial-XXXX
```

6.3 Capteur de pression atmosphérique ST LPS22HB

La carte comporte un capteur ST LPS22HB de mesure de la température et de la pression atmosphérique [6]. Le pilote générique de cette gamme de capteurs LPSxxx est fourni par RIOT OS.

```
cd ~/github/RIOT-OS/RIOT/tests/drivers/lpsxxx/
DRIVER=lps22hb make -j 4 flash
```

Console :

```
main(): This is RIOT! (Version: 2023.07-devel-325-g2863d)
Test application for lps22hb pressure sensor
Initializing lps22hb sensor
Pressure value: 995hPa - Temperature: 21.21°C
Pressure value: 995hPa - Temperature: 21.20°C
...
Pressure value: 869hPa - Temperature: 30.34°C
```

6.4 Accéléromètre LIS2DE12

La carte comporte un accéléromètre ST LIS2DE12 pilotable via le bus I2C [6] . RIOT fournit un driver pour un accéléromètre similaire (lis2dh12).

```
cd ~/github/RIOT-OS/RIOT/tests/drivers/lis2dh12/
DRIVER=lis2dh12_i2c make -j 4 flash
```

Console :

```
main(): This is RIOT! (Version: 2024.01-devel-259-ga38ab)
using I2C mode, for SPI mode select the lis2dh12_spi module
lis2dh12 [Initialized]
Saiu do init ........................
Shell init ........................
> help
Command              Description
---------------------------------------
read                 Read acceleration data
read_fifo            Read acceleration data from fifo
threshold            Configure threshold event
click                Configure click event
power                Enable / Disable the sensor
resolution           Get/Set resolution
rate                 Get/Set sampline rate
scale                Get/Set measuring range
temp                 Read temperature data
read_int             Read interruption data
pm                   interact with layered PM subsystem
reboot               Reboot the node
version              Prints current RIOT_VERSION
```

Il faut Changer la position de la carte entre chaque commande `read`.

6.5 Capteurs I2C externes

La carte peut être étendue en raccordant un ou plusieurs capteurs I2C via le connecteur Grove I2C de la carte [6]. Il faut repérer le modèle de capteur qu'on possède et le brancher au connecteur Grove I2C de la carte. Il faut respecter la couleur des fils et des broches pour ne pas endommager les composants et la carte [6].

Plusieurs capteurs I2C externes peuvent être raccordés à la carte Wyres grâce à la platine I2C Hub [6].

Broche Platine	Fil Grove
VCC	Rouge
GND	Noir
SDA	Blanc
SDL	Jaune

Table 3: Caption

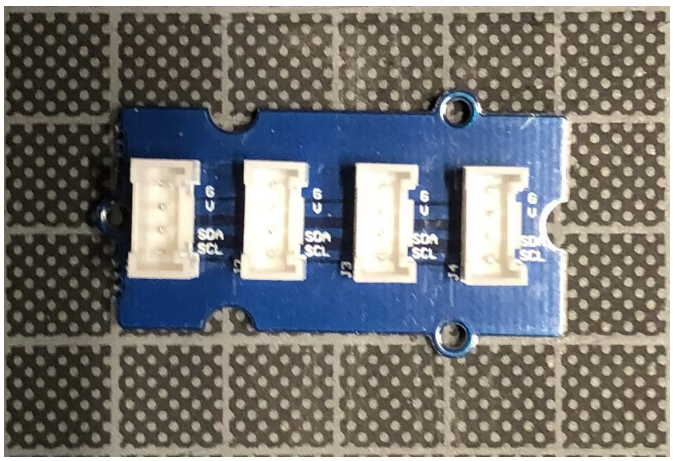


Figure 7: Platine I2C Hub

6.6 BME280

Le composant Bosch Sensortec BME280 mesure la température ambiante, la pression atmosphérique et l'humidité relative [6]. Le pilote générique de cette gamme de capteurs bme280 est fourni par RIOT OS.

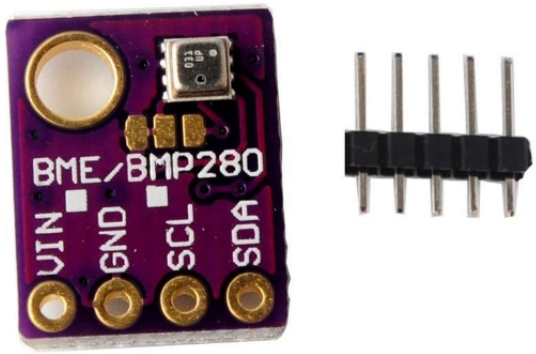


Figure 8: BME280

```
cd ~/github/RIOT-OS/RIOT/tests/drivers/bmx280/
make -j 4 flash
```

6.7 BME680

Le composant Bosch Sensortec BME680 mesure la température ambiante, la pression atmosphérique, l'humidité relative et la concentration en Composés Organiques Volatiles (COV) [6]. Le pilote de ce capteur bme680 est fourni par RIOT OS.

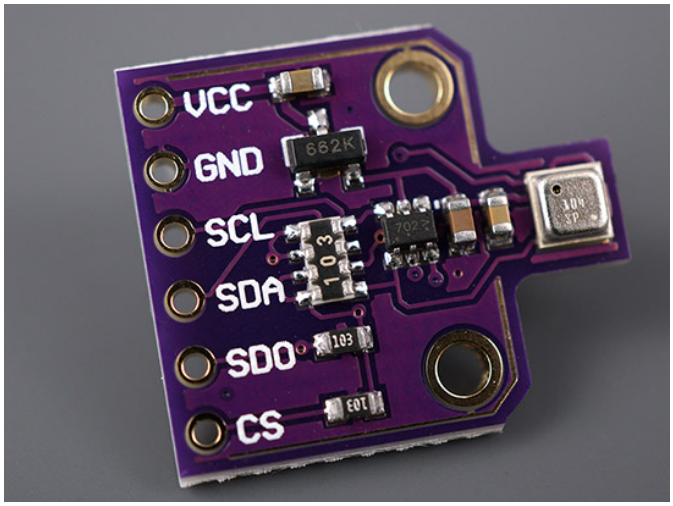


Figure 9: BME680

```
cd ~/github/RIOT-OS/RIOT/tests/drivers/bme680/
USEMODULE='bme680_fp bme680_i2c printf_float' make -j 4 flash
```

Console :

```
Help: Press s to start test, r to print it is ready
START
main(): This is RIOT! (Version: 2023.07-devel-325-g2863d)
Initialize BME680 sensor 0 ... OK
[bme680]: dev=0 T = 27.04 degC, P = 86734.91 Pa, H 63.732 %, G = 31305 ohms
+----------------------------------------+
[bme680]: dev=0 T = 27.05 degC, P = 86735.34 Pa, H 63.672 %, G = 29744 ohms
```

L'adresse I2C est 0x77.

6.8 MPL3115A2

Le composant NXP MPL3115A2 mesure la température ambiante, l'altitude (via la pression atmosphérique) et l'humidité relative [6]. Le pilote de ce capteur mpl3115a2 est fourni par RIOT OS. Les plages de mesure et les précisions sont:

- Pression: 20 à 110 kPa sur 20 bits
- Altitude: –698 à 11,775 m sur 20 bits
- Température: –40 °C à 85 °C sur 12 bits

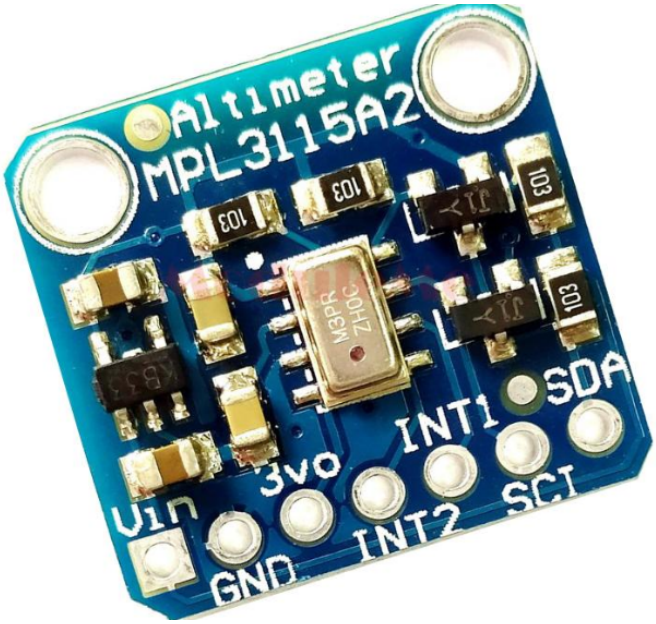


Figure 10: MPL3115A2

```
cd ~/github/RIOT-OS/RIOT/tests/drivers/mpl3115a2/
make -j 4 flash
```

7 Communications LoRa

Commençons par expliquer le concept LoRa (Long Range), une technologie de communication sans fil développée pour les applications IoT (Internet des Objets) nécessitant une transmission longue portée, une consommation d'énergie réduite et une connectivité à faible débit [7]. La communication avec LoRa se fait généralement à travers des réseaux LoRaWAN ou via des communications point à point LoRa.

Quant au terme LoRaWAN, il signifie Long Range Wide Area Network ou réseau étendu à longue portée. Il s'agit d'un protocole radio de télécommunication permettant la transmission de petits paquets de données à bas débit, particulièrement adapté aux objets connectés[7]. L'émission du signal radio se fait sur un large spectre afin d'offrir une robustesse aux interférences et aux signaux parasites. Cette technologie offre une grande simplicité de communication et permet des échanges de données sur une longue portée.

Dans ce chapitre, vous apprendrez à transmettre et recevoir des informations en utilisant directement le pilote des transceivers LoRa SX1272 de 2 (ou plus) cartes Wyres Base (la vôtre et celles de vos voisins).

7.1 Rappel de quelques notions

Comme vous l'avez vue dans les chapitre 1 et 2 pour configurer la carte cible pour make, copier et exécuter ses commandes suivante sur votre terminal :

7.1.1 Configuration du développement

Pour configurer votre environnement de développement, vous pouvez définir les variables d'environnement suivantes :

```
export BOARD=wyres-base
export EXTERNAL_BOARD_DIRS=~/github/campusiot/RIOT-wyres/boards
```

Recherchez le tty de la console et connectez-vous à celle-ci avec tio pour cela vous devez tout d'abord Branchez votre périphérique CH340 du port USB.

7.1.2 Instructions pour Linux

Si vous êtes sur Linux, suivez ces étapes :

```
lsusb
tio
tio -b 115200 -m INLCRNL /dev/ttyUSB0
```

Il est important de taper chaque commande l'une après l'autre pour vérifier si elles s'exécutent sans générer d'erreurs. En cas d'erreur, vous pourrez ainsi déterminer quelle commande l'a causée.

7.1.3 Instructions pour Mac OS

Si vous êtes sur Mac OS, voici les commandes à exécuter :

```
lsusb
tio
tio -L
tio -b 115200 -m INLCRNL /dev/tty.usbserial-XXXX
```

De même, exécutez ces commandes une par une pour vérifier leur exécution sans erreur. En cas d'erreur, cela facilitera l'identification de la commande à l'origine de l'erreur.

7.1.4 Notes supplémentaires

NB : Pour mémoire, si vous utilisez Linux dans une machine virtuelle VirtualBox, il faut monter les 2 périphériques USBSerial et STLink dans le menu Péripériques de la machine virtuelle [7].

7.2 Les éléments les plus importants dans la communication LoRa

Dans ce chapitre ou on parle de la communication LoRa voici les éléments les plus importantes de cette communication. On a entre autre :

7.2.1 La modulation de communication LoRa

La modulation de communication LoRa utilise une technique brevetée de modulation par étalement de spectre (de type Chirp Spread Spectrum)[7].

La modulation de communication LoRa cible principalement les communications nécessitant de faire transiter un volume de données faible avec un débit réduit, ce qui permet de limiter la consommation énergétique des équipements connectés et de proposer des autonomies de plusieurs années sur piles[7].

La modulation de communication LoRa s'utilise sur des bandes de fréquences libres (utilisables sans déclaration et sans redevance) permettant de réduire les coûts de fonctionnement du réseau[7]. Elle offre ainsi la possibilité de créer des réseaux dits privés (indépendants des opérateurs) pouvant couvrir un immeuble, un champ ou une ville sans frais d'abonnement.

7.2.2 Communication point-à-point (P2P) avec LoRa

RIOT autorise l'utilisation direct des pilotes des périphériques de communication (Ethernet, 802.15.4 ...) sans passer par des couches d'abstraction réseau (IP, LoRaMAC ...). Dans le cas de la carte Wyres Base, le périphérique de communication radio est le composant Semtech SX1272 qui permet d'écouter des messages transmis en modulation LoRa ou GFSK et de transmettre des messages avec modulation LoRa ou GFSK en consommant peu d'énergie [7]. L'écoute est exclusive de la transmission.

Le programme suivant démontre la configuration des paramêtres de communication, la transmission de messages et la réception de messages.

Pour les tests, accordez vous avec votre voisin sur des paramêtres de communication pour échanger avec lui des messages LoRa. Il conviendra d'utiliser des canaux de fréquence qui sont autorisés par la réglementation locale (entre 863 et 870 MHz en Europe par exemple).

7.2.3 Commandes pour le développement

Pour effectuer le développement avec RIOT et les transceivers SX127x, vous pouvez suivre ces étapes :

```
cd ~/github/campusiot/RIOT-wyres/tests/driver_sx127x
make -j 4 flash
```

Ces commandes changent le répertoire vers le dossier des tests du driver SX127x de RIOT et lancent la compilation avec 4 threads (‘-j 4‘) suivie de la mise à jour du firmware (‘flash‘).

Vous obtiendrez le résultat suivant comme le montre cette figure



Figure 11: Console tio

7.2.4 LoRaChat

Utilisez le programme précédent pour élaborer un chat en LoRa [7]. Il suffit pour cela d'établir une convention de rédaction des messages. Par exemple :

- `123@*:11:Qui est là ?` : signifie de la part de 123 pour tout le monde (*) (message #11), "Qui est là ?" – doit entraîner une réponse des récepteurs.
- `123@*:12:Salut à tous` : signifie de la part de 123 pour tout le monde (*) (message #12).
- `123@456:13:Salut au groupe 456` : signifie de la part de 123 pour 456 (message #13).
- `123@12*:14:RDV 868100000 SF7BW125` : signifie de la part de 123 pour les groupes dont le numéro commence par 12* (message #14), RdV sur le canal 868.1 MHz en SF7 et BW125.
- `123@*:15:SOS 3.86292,11.50003` : signifie de la part de 123 pour tout le monde (*), SOS, je suis à la position 3.86292,11.50003.

Ces conventions permettent de structurer les échanges dans le chat LoRa en spécifiant l'expéditeur, le destinataire (ou tous les destinataires), le numéro de message, et le contenu du message avec des informations supplémentaires si nécessaire.

7.2.5 Observation depuis les stations

Ouvrez la console Web d'une des stations de l'organisation pour observer les échanges sur les canaux 868.1 MHz, 868.3 MHz, 868.5 MHz, 867.1 MHz, 867.3 MHz, 867.5 MHz, 867.7 MHz, 867.9 MHz (SF7 à SF12 et BW125).

Observez-vous dans la console Web des gateways, les échanges sur les canaux 868.0 MHz, 868.2 MHz, 868.4 MHz, 867.0 MHz, 867.2 MHz, ... (SF7 à SF12 et BW125, BW250 et BW500) ?

8 Communications LoRa (avancé)

L'objectif de ce chapitre, vous apprendrez à transmettre et recevoir des informations vers des applications sur des serveurs en utilisant les services d'une infrastructure LoRaWAN.

8.1 L'infrastructure LoRaWAN

Les réseaux LoRaWAN ont une topologie dite en étoile [8]. Les équipemnts ne dialoguent qu'avec les stations de base (cf. figure de gauche). Les équipements ne dialoguent pas entre-eux comme dans les réseaux maillés (cf. figure à droite).

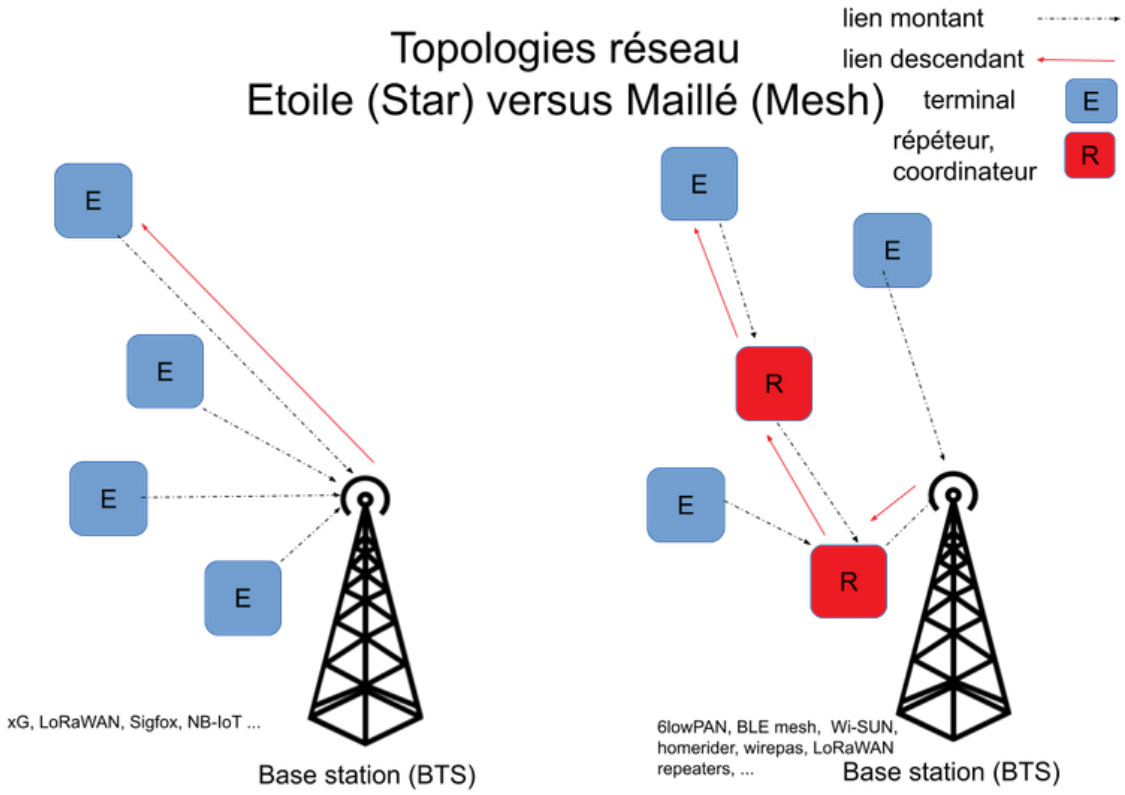


Figure 12: Topologie des reseau LoRaWAN

Dans un réseau LoRaWAN, toutes les stations de base écoutent (et transmettent) sur les mêmes canaux d'une bande de fréquence[8]. De ce fait, un message émis par un équipement peut être reçu par une à plusieurs stations. Le serveur du réseau est en charge de dédupliquer les duplicats d'un même message. Dans le cas où un équipement est entendu par plusieurs stations, la panne d'une d'entre elles ne crée par d'interuption de la collecte des mesures (cf. le schéma suivant).

8.2 Les éléments

Les équipements d'un réseau LoRaWAN [8] sont :

- Terminal (Endpoint ou Device en anglais)
- Point d'accès ou Station de Base ou Passerelle (Gateway en anglais)
- Join Server
- Network Server
- Application Server

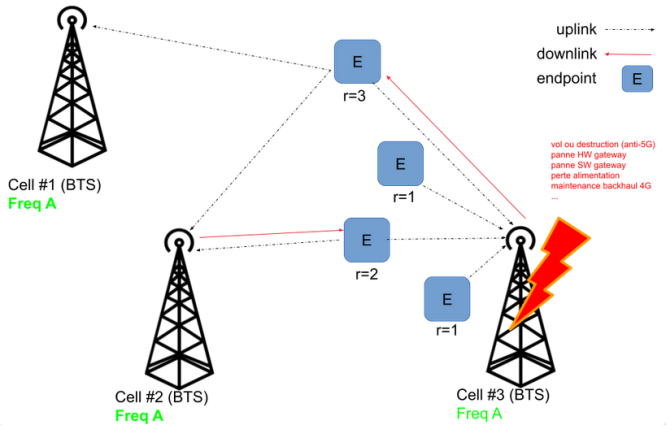


Figure 13: Illustration du cas de panne d'un équipement

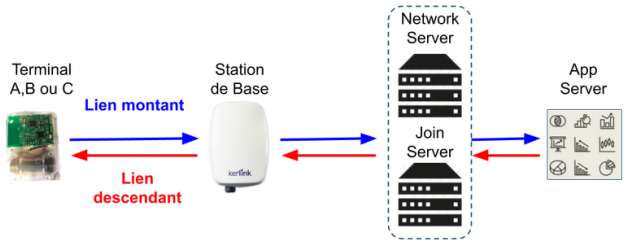


Figure 14: Les équipements d'un réseau LoRaWAN

Ces éléments sont réprésenté à la figure si après.

Ces différents composants jouent des rôles spécifiques dans l'infrastructure d'un réseau LoRaWAN, permettant ainsi la gestion des périphériques connectés, le routage des données et les interactions avec les applications finales.

8.3 Les identifiants et les clés

L'activation (OTAA) utilise les informations suivantes pour authentifier un équipement auprès d'un réseau d'opérateur :

- L'identifiant unique d'équipement DevEUI, dont le préfixe est normalement l'identifiant unique de fabricant (OUI).
- AppEUI / JoinEUI, qui sert aux équipements vagabonds (roaming en anglais).
- AppKey, la clé dite applicative qui sert à authentifier l'équipement auprès du serveur du réseau et qui sert à authentifier le serveur du réseau dans lequel l'équipement est censé être enregistré par son propriétaire.

À l'issue de la procédure d'activation, l'équipement reçoit un identifiant dans le réseau, le DevAddr [8]. Cet identifiant est normalement pris dans une plage d'adresse affectée à un opérateur LoRaWAN par la LoRa Alliance. Cette plage se calcule à partir du NetId. Deux NetId sont réservés aux réseaux expérimentaux (00, 01).

La suite des échanges est sécurisée au moyen de deux clés : la clé de session réseau NwkSKey qui sert à signer (et donc autentifier) les messages et la clé de session applicative AppSKey qui sert à chiffrer (donc rendre confidentiel) la charge utile (ou applicative) du message.

Notes sur les NetIds à Grenoble

Note : Sur Grenoble, les NetIds que vous pourrez observer le plus fréquemment dans la console des stations sont :

- 000007 pour Objenious racheté par NetMore (DevAddr compris entre 0E000000 et 0FFFFFFF).

- 00000F pour LiveObject (Orange) (DevAddr compris entre 1E000000 et 1FFFFFFF).
- 000013 pour The Things Network (DevAddr compris entre 26000000 et 27FFFFFF).
- C0002B pour l'Université Grenoble Alpes (DevAddr compris entre FC00AC00 et FC00AFFF).

8.4 Classes d'objet dans LoRaWAN

Les classes d'objet définissent le comportement des terminaux LoRaWAN [8] :

- Classe A : pour un terminal autonome en énergie (batteries). Réception du message descendant (downlink) 1 seconde exactement après la fin de la transmission d'un message montant.
- Classe B : pour un terminal autonome en énergie (batteries). Écoute périodique (8 fois / 128 secondes) d'un canal pour la réception du message descendant (downlink). Synchronisation régulière de l'oscillateur du terminal avec un "beacon" pour corriger sa dérive.
- Classe C : pour un terminal autonome en énergie (batteries). Écoute permanente d'un canal pour la réception du message descendant (downlink).
- Classe C Multicast : pour procéder notamment à la mise à jour d'un parc d'équipement.

9 Collecte,stockage et visualisation des données des capteurs LORaWAN

Dans ce chapitre nous allons apprendre à :

- Intégrer les mesures de capteurs LoRaWAN dans des plateformes IoT et Cloud
- Collecter, stocker et afficher les mesures de capteurs LoRaWAN dans des applications on-premise
- Utiliser des archives de mesures de capteurs LoRaWAN pour des analyses

9.1 Nodered - Influxdb - Grafana (on premise) : Collecte, stockage et affichage des mesures de capteurs LoRaWAN

Dans cette leçon, vous apprendrez à collecter, stocker et afficher les mesures de capteurs LoRaWAN avec la composition docker Nodered - Influxdb - Grafana.

La chaîne de traitement des mesures est constituée des services dockerisés [10] suivants :

- nodered pour la récupération des messages en souscrivant au broker MQTT
- influxdb pour stocker les mesures dans des séries temporelles
- grafana pour afficher dans des tableaux de bord les séries temporelles stockées par le service influxdb

9.1.1 Lancement de la composition

Lancez les commandes suivantes :

```
cd ~/github/campusiot/RIOT-wyres/backend
docker-compose up -d
docker-compose ps
docker-compose logs -f
```

9.1.2 Collecte et stockage

Ouvrez la console Web du service NodeRed (admin CHANGE_THIS_UNSECURE_PASSWORD)

Configurez dans le nœud MQTT le broker MQTT LNS (lns.campusiot.imag.fr) en ajoutant l'identifiant et le mot de passe pour se connecter au broker MQTT (l'identifiant et le mot de passe vous seront communiqués en séance) [10]. Le service nodered est en charge de la récupération des messages (onglet Collect Devices) et du stockage sous la forme de séries temporelles (en anglais time series) dans le service influxdb (onglet Save Data).

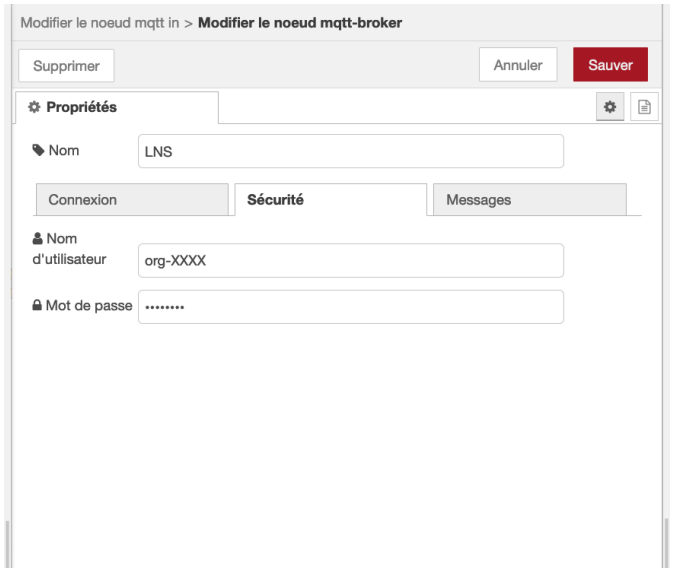


Figure 15: Interface NodeRed 1

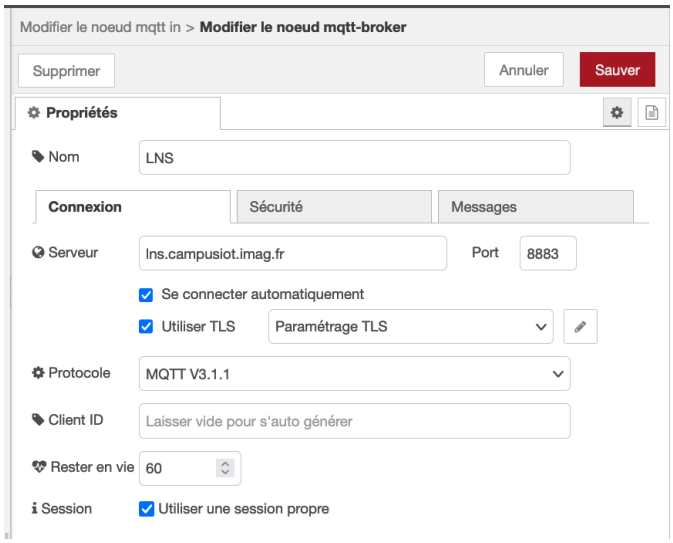


Figure 16: Interface NodeRed 2

9.1.3 Visualisation

Le service grafana est en charge de la visualisation

Ouvrez la console Web du service grafana (admin CHANGE_THIS_UNSECURE_PASSWORD).

Un certain nombre de tableaux de bord ont été pré-installés.

Vous pouvez éditer/compléter/transformer les tableaux de bord existants à votre guise.

9.1.4 Ajout d'un panel

Ajoutez un panel pour afficher le taux de perte des paquets LoRaWAN (PER pour Packet Error Ratio en anglais) pendant l'intervalle de temps sélectionné.

9.1.5 Envoi de messages vers les équipements

Retournez dans la console Web du service nodered.

Ouvrez l'onglet Downlink : il montre comment un message descendant peut être envoyé à un équipement LoRaWAN (classe A, B ou C) via l'intégration MQTT de Chirpstack.



Figure 17: Interface Manipulation carte Wyres Base

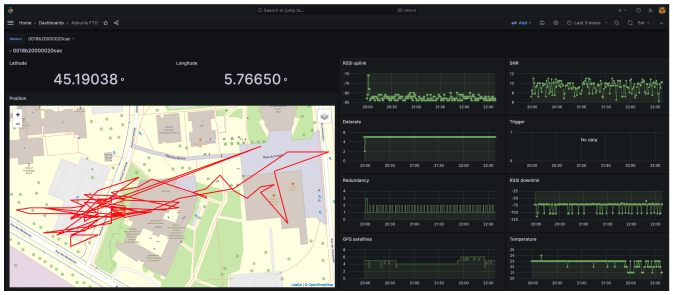


Figure 18: Interface manipulation carte Wyres Base 2

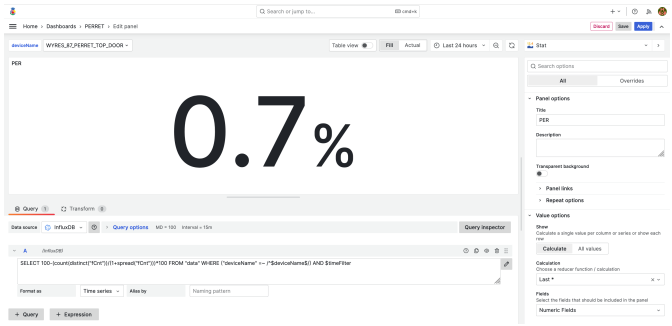


Figure 19: Presentation interface ajout panel

Cliquez sur le nœud 'horodatage'. Vérifiez dans la console de l'équipement que le message est dans la file

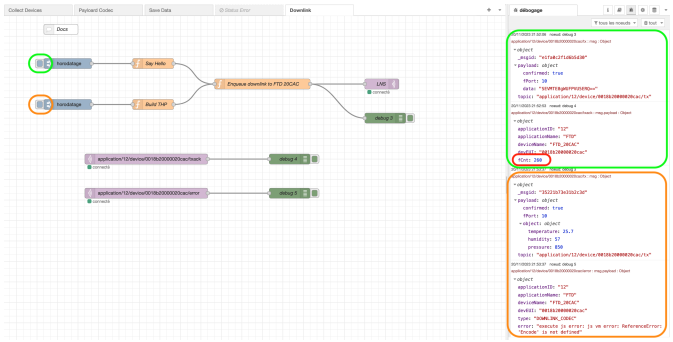


Figure 20: Presentation schema des messages echangees

Downlink Queue. A la réception d'un message montant, le message mis en queue des messages descendants est transmis vers l'équipement.

Dans le cas du Testeur de réseau de terrain (FTD) d'Adeunis, le message est affiché sur son écran LCD comme ci-dessous.

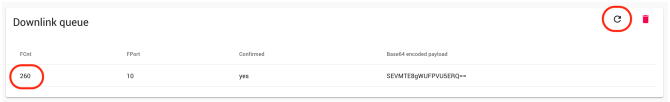


Figure 21: Presentation du message Downlink



Figure 22: FabMSTIC 20cac

9.1.6 Inspection de la base de données InfluxDB

Vous pouvez inspecter la base de données InfluxDB avec les lignes de commande suivantes :

```
docker-compose exec influxdb influx
> help
> show databases
> use lorawan
> show tag keys
> show field keys
> show series
> show measurements
> select * from data limit(10);
> select * from radio limit(10);
> select * from data where time > now() - 1h
> select * from "data" where "deviceName" =~ /Wyres_.*/
> SELECT * FROM "data" WHERE "deviceName" =~ /Wyres_.*/ AND time > now() - 1h
> SELECT count(*) FROM "data" WHERE "deviceName" =~ /Wyres_.*/ AND time > now() - 6h GROUP BY time(10m
> SELECT mean("temperature") as mean_temperature, mean("temperature") as mean_pressure, mean("light")
```

9.1.7 Export des séries temporelles depuis la base de données InfluxDB

Une série temporelle peut être exportée dans un fichier avec les lignes de commande suivantes. Au format CSV :

```
QUERY='SELECT * FROM "data"'
docker-compose exec influxdb influx -database lorawan -execute "$QUERY" -format csv > ./export.csv
wc -l ./export.csv
head ./export.csv
```

9.1.8 Import d'une série temporelle dans la base de données InfluxDB

Lisez la documentation et csv-to-influxdb.

9.1.9 Qu'est-ce qu'InfluxDB ?

InfluxDB est un système de gestion de base de données développé par la société InfluxData, Inc [9]. InfluxDB est un logiciel open source et peut être utilisé gratuitement. La version commerciale InfluxDB Enterprise inclut un accord de maintenance ainsi que des contrôles d'accès spéciaux pour les clients professionnels et s'installe sur un serveur du réseau de l'entreprise [10].

La nouvelle version InfluxDB 2.0 offre en plus un service cloud flexible et adaptable et comprend une interface utilisateur web pour saisir et visualiser les données.

Le système de gestion de base de données InfluxDB utilise le langage de programmation Go de Google, encore appelé Golang. Dans la première version, le langage de requête InfluxQL, développé par la société, était utilisé pour interroger les bases de données externes.

InfluxDB 2.0 introduit le nouveau langage de programmation Flux, qu'InfluxData publie sur GitHub en open source sous licence MIT. Là-bas, d'autres développeurs manipulant des données chronologiques continueront d'améliorer le projet. Flux est un langage de script et de requête indépendant destiné aux bases de données de série temporelle (TSDB)[9]. Depuis la version 1.7, il peut être utilisé de manière entièrement indépendante ou en combinaison avec des bases de données d'autres fournisseurs.

Flux est optimisé pour le processus ETL (Extract, Transform, Load) mais n'est pas rétrocompatible avec le langage de requête précédemment utilisé, InfluxQL. Néanmoins, la société prévoit un chemin de migration pour les clients existants, qui devrait convertir le code InfluxQL en Flux.

La syntaxe de Flux est basée sur le langage de script JavaScript, ce qui signifie qu'il est facile à apprendre et librement extensible [9]. Une caractéristique majeure de Flux est la prise en charge de différentes sources de données, par exemple via des API de fournisseurs tiers. Flux peut donc travailler avec des outils d'analyse tels que Jupyter.

L'interface d'échange de données Apache Arrow permet une communication avec d'autres systèmes et une intégration aux environnements big data.

9.1.10 Quand utiliser InfluxDB ?

InfluxDB est conçu pour gérer les bases de données de type TSDB (time series database), qui contiennent des séries temporelles [9]. Ces bases de données sont employées notamment pour stocker et analyser des données de capteurs ou des journaux horodatés sur une période donnée.

Il peut être question de millions d'enregistrements entrants, comme les objets connectés de l'internet des objets ou les instruments de mesure scientifiques en fournissent en flux continu.

Ces données doivent être traitées rapidement une fois entrées dans la base de données. C'est pourquoi InfluxDB intègre un service de temps qui, grâce au Network Time Protocol (NTP), assure que l'heure est bien synchrone sur l'ensemble des systèmes [9].

Dans InfluxDB, une base de données peut être très compacte et doit comporter deux ou trois colonnes seulement. Dans l'exemple ci-dessous, on trouve la source de données, la valeur mesurée et la date/heure.

Capteur	Valeur	Temps
Capteur 1	140.50	23/04/2020 @ 10:00
Capteur 2	110.02	23/04/2020 @ 10:00
Capteur 1	142.32	23/04/2020 @ 10:05
Capteur 2	110.50	23/04/2020 @ 10:05

InfluxDB opère une distinction entre **tags** et **fields** concernant les colonnes. Alors qu'un **tag** contient seulement des métadonnées qui sont indexées, les **fields** contiennent des valeurs qui pourront être analysées par la suite. Dans notre exemple, la première colonne est un **tag**, tandis que la deuxième est un **field**. Cette distinction simplifie l'administration de la base de données et l'analyse des données de mesure.

9.1.11 Sauvegarde/Restauration d'une base de données InfluxDB

- Sauvegarde

```
BACKUP_DIR=/var/lib/influxdb/backup/backup-$(date +%Y%m%d)
docker-compose exec influxdb bash -c "mkdir -p $BACKUP_DIR && influxd backup -portable -db lorawa
```

- Restauration

```
BACKUP_DATE=...
BACKUP_DIR=/var/lib/influxdb/backup/backup-$BACKUP_DATE
docker-compose exec influxdb bash -c "influxd restore -portable -db lorawan $BACKUP_DIR"
```

Remarques

- Le mot de passe Grafana se change dans le fichier `grafana.yml`.
- Le mot de passe Nodered se change dans le fichier `settings.js` après hachage.
- Le service `nodered` peut être complété avec d'autres types de nœud via le `Dockerfile` du service `nodered`.
- Le service `grafana` peut être complété avec d'autres plugins via la définition `grafana.yml` du service `grafana`.
- NodeRED intègre la possibilité de créer des tableaux de bord qui peuvent contenir des actions et même des applications plus complexes avec NodeRed UI Builder.
- N'hésitez pas à jeter un coup d'oeil aux exemples de flux Nodered contribués par la communauté OpenJS.

9.2 Thingsboard (cloud) : Collecte, stockage et affichage des mesures de capteurs LoRaWAN.

Dans cette leçon, vous apprendrez à collecter, stocker et afficher les mesures de capteurs LoRaWAN avec le service cloud de Thingsboard.

9.2.1 Création d'un compte Thingsboard

Créez-vous un compte sur le service cloud Thingsboard. ThingsBoard.cloud crée une instance qui vous est dédiée. Ça peut prendre quelques minutes. Cette instance d'évaluation est gratuite pendant un mois. Connectez-vous à votre instance ThingsBoard

Pour ajouter votre équipemnent à Thingsboard, accédez à la section "Entités"[11]. Ensuite, allez à la page "Dispositifs".

Cliquez sur l'icône "+" en haut à droite du tableau, puis sélectionnez "Ajouter un nouvel dispositif". Notez l'Access Token du dispositif et terminez la création en cliquant sur "Ajouter" ("Add"). Testez l'envoi d'une mesure factive (aka télémétrie) pour tester la configuration :

```
OBJECT\_JSON='{"temperature":25.0,"pressure":995,"humidity":75,"battery":3200,"hvacEnabled":true,"hvacStat
ACCESS\_TOKEN=XXXXXXXXXXX
curl -v -X POST http://thingsboard.cloud/api/v1/$ACCESS_TOKEN/telemetry --header Content-Type:application/
```

Dans la console du LNS, ajoutez une nouvelle intégration Thingsboard depuis l'onglet INTEGRATIONS votre application.

Thingsboard

L'adresse du serveur a spécifié est https://thingsboard.cloud.

Dans l'onglet VARIABLES de l'onglet CONFIGURATION de votre équipement, ajoutez une nouvelle variable ThingsBoardAccessToken avec la valeur de l'Access Token donnée par Thingsboard précédenment [11].

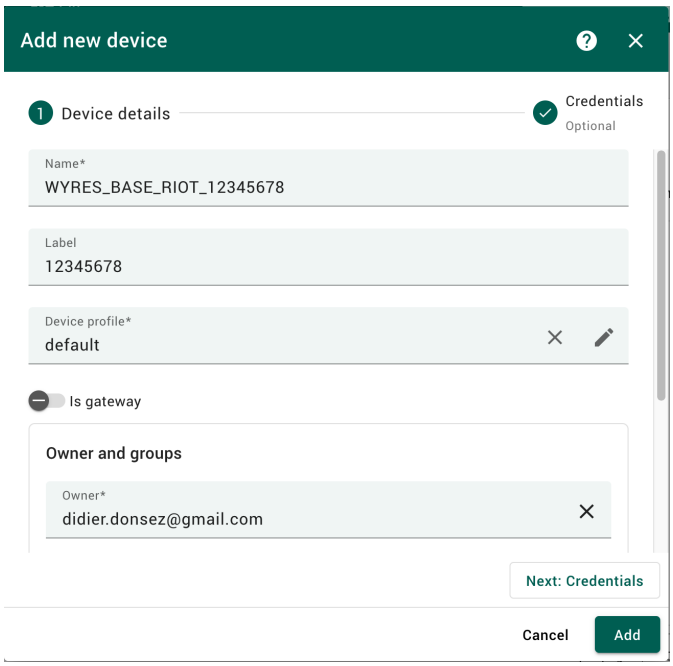


Figure 23: Interface ajout device type Wyres Base 1

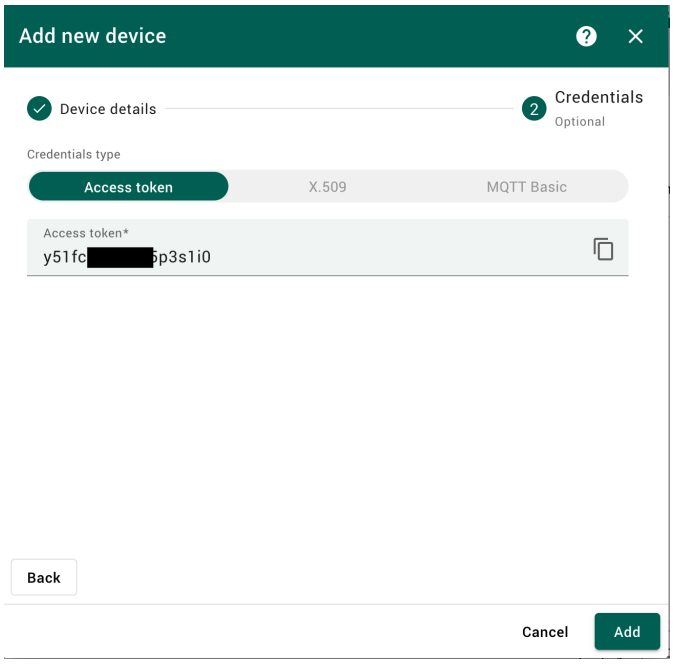


Figure 24: Interface ajout device type Wyres Base 2

Désormais que le LNS reçoit un message de l'équipement, il envoit vers Thingsboard une réquête HTTPS contenant les nouvelles mesures contenues dans la charge utile (décodée)ndu message.

Vous pouvez désormais ajouter un nouveau tableau de bord à votre instance Thingsboard en cliquant sur l'icône "+".

Entrez dans le mode Edition pour ajouter un nouveau Widget (que vous choissirez parmi la liste proposée en fonction de vos goûts) et sélectionnez le dispositif (parmi les dispositifs déjà créés) et la mesure (parmi les mesures (aka Data Keys) déjà reçues). N'oubliez pas de sauvegarder le tableau de tableau. Plus de détail sur l'intégration Thingsboard de Chirpstack.

Vous pouvez approfondir l'utilisation de Thingsboard par vous-même pour créer des alarmes sur des mesures

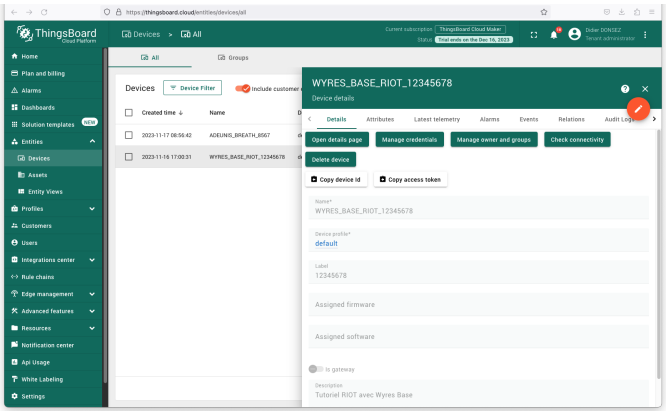


Figure 25: Interface Thingboard

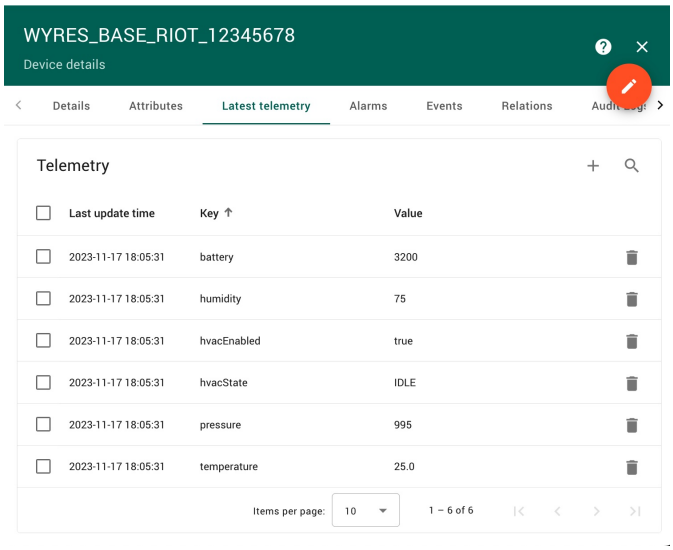


Figure 26: Interface presentation activite device Wyres Base 12345678

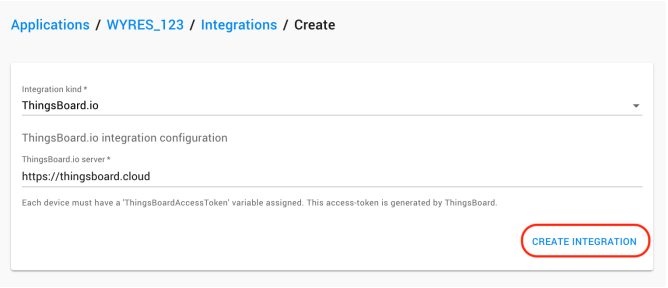


Figure 27: Interface creation device

envoyées par les équipements (par exemple, dépassement d'un seuil de température d'une chambre froide) et des règles entrainant l'exécution d'actions.

Alarmes Chaines de règles

9.2.2 Configuration de l'équipement

1. Connectez-vous à votre instance ThingsBoard.
2. Pour ajouter votre équipement à Thingsboard, accédez à la section "Entités". Ensuite, allez à la page

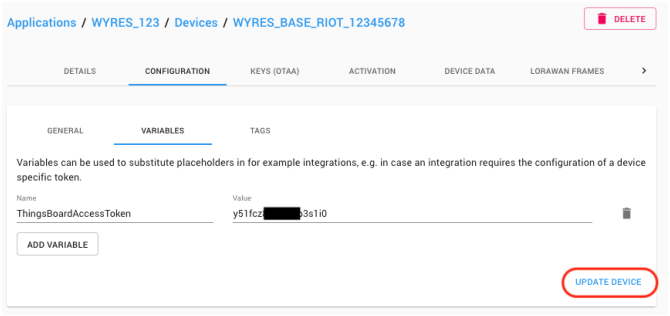


Figure 28: Interface configuration device Wyres Base Riot 12345678

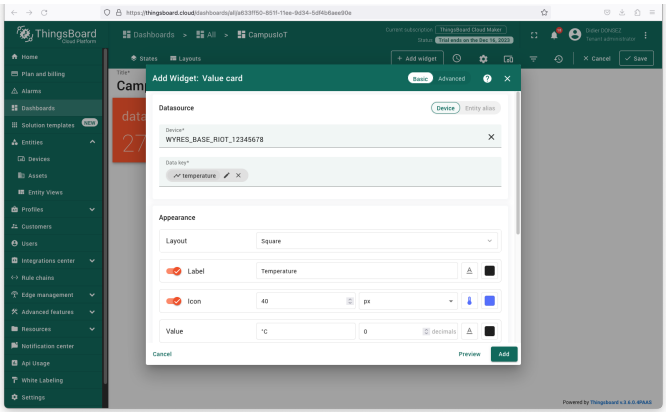


Figure 29: Interface ThingBoard ajout Widget

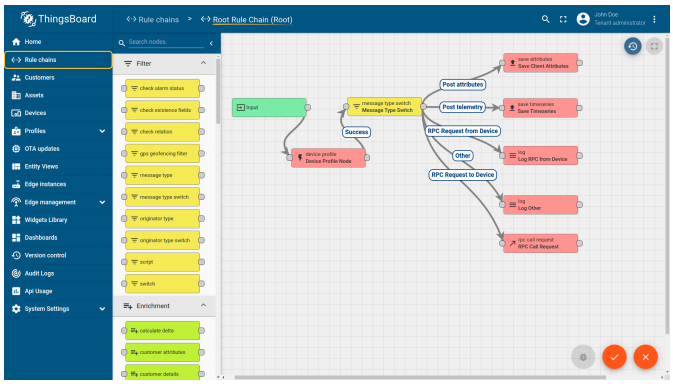


Figure 30: Interface ThingBoard; presentation des Root Rule Chain

"Dispositifs".

3. Cliquez sur l'icône "+" en haut à droite du tableau, puis sélectionnez "Ajouter un nouvel dispositif".
4. Entrez le nom de l'équipement. Par exemple, "WYRES_BASE_RIOT_12345678" et un label (par exemple, le texte imprimé sur l'étiquette du boitier de l'équipement).
5. Notez l'Access Token du dispositif et terminez la création en cliquant sur "Ajouter" ("Add").
6. Testez l'envoi d'une mesure factice (aka télémétrie) pour tester la configuration :

```
OBJECT\_JSON='{"temperature":25.0,"pressure":995,"humidity":75,"battery":3200,"hvacEnabled":true,"hvac
ACCESS\_TOKEN=XXXXXXXXXXX
curl -v -X POST http://thingsboard.cloud/api/v1/\$ACCESS\_TOKEN/telemetry --header Content-Type:applic
```

9.2.3 Configuration de l'intégration dans le LNS

Dans la console du LNS, ajoutez une nouvelle intégration Thingsboard depuis l'onglet INTEGRATIONS votre application.

9.3 Thingsboard (on premise) : Collecte, stockage et affichage des mesures de capteurs LoRaWAN

Dans cette partie, vous apprendrez à collecter, stocker et afficher les mesures de capteurs LoRaWAN dans Thingsboard on premise. Thingsboard est également disponible sous la forme de conteneur Docker.Cependant, l'intégration Thingsboard dans le LNS n'est pas possible puisque votre machine ne dispose d'une adresse publique [12]. Nodered sera en charge d'effectuer cette intégration vers le service Docker thingsboard. La composition backend contient déjà Nodered-InfluxDB-Grafana. Décommentez le service thingsboard dans le fichier docker-compose.yml. Redémarrez la composition avec les commandes suivantes:

- cd /github/campusiot/RIOT-wyres/backend
- docker-compose down
- docker-compose up -d
- docker-compose ps
- docker-compose logs -f

Ouvrez la console web du service Thingsboard. Vous pouvez utiliser les 3 comptes par défaut :

- Systen Administrator: sysadmin@thingsboard.org / sysadmin
- Tenant Administrator: tenant@thingsboard.org / tenant
- Customer User: customer@thingsboard.org / customer

Vous pouvez toujours changer les mots de passe pour chaque compte dans la page de profil du compte. Connectez-vous avec le compte Tenant : tenant@thingsboard.org / tenant Comme dans le cas de l'instance cloud de Thingsboard [12], ajoutez le dispositif et recupérez l'Access Token. Renseignez l'Access Token dans la variable ThingsBoardAccessToken dans l'onglet VARIABLES de l'onglet CONFIGURATION de votre équipement. Modifiez le codec dans l'onglet CODEC du Device Profile de l'équipement pour ajouter les variables (qui contiennent la variable ThingsBoardAccessToken) dans l'objet decodé. Par exemple:

```
// Decode decodes an array of bytes into an object.
//  - fPort contains the LoRaWAN fPort number
//  - bytes is an array of bytes, e.g. [225, 230, 255, 0]
//  - variables contains the device variables e.g. {"calibration": "3.5"} (both the key / value are of
// The function must return an object, e.g. {"temperature": 22.5}
function Decode(fPort, bytes, variables) {
    var result = "";
    for(var i = 0; i < bytes.length; ++i){
        result+= (String.fromCharCode(bytes[i]));
    }
    var decoded = { text: result }
    if(variables.length > 0) {
        decoded._variables = variables;
    }
    return decoded;
}
return decoded;
}
```

Modifiez le flot Nodered pour que les message de l'équipement reçus vers MQTT soit transformé en requête HTTP vers le service docker thingsboard. La requête est équivalente à la commande cURL suivante:

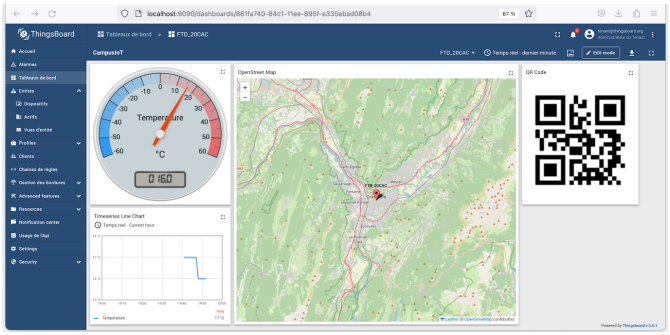


Figure 31: Interface ThingBoard CalpusloT

```
OBJECT_JSON='{"temperature":25.0,"pressure":995,"humidity":75,"battery":3200,"hvacEnabled":true,"hvacS
ACCESS_TOKEN=XXXXXXXXXXXX
curl -v -X POST http://thingsboard:9090/api/v1/$ACCESS_TOKEN/telemetry --header Content-Type:applicati
```

Poursuivez par la création d'un tableau de bord à votre goût, l'ajout d'alarmes et de règles.

9.4 Influx Data (cloud) : Collecte, stockage et affichage des mesures de capteurs LoRaWAN.

Dans cette leçon, vous apprendrez à collecter, stocker et afficher les mesures de capteurs LoRaWAN dans le service cloud InfluxData.

InfluxDB est une base de données temporelle spécialisée dans le stockage de mesures IoT [13]. Elle peut être opérée soi-même sur ses propres serveurs (on-premise). Plusieurs fournisseurs cloud offrent un service de stockage InfluxDB sur leur cloud. InfluxData est l'un d'entre eux.

Chirpstack permet une intégration avec un serveur InfluxDB : `https://www.chirpstack.io/application-server/integrations/influxdb/`

Créez-vous un compte gratuit sur le service de stockage en nuage d'InfluxData (une carte de crédit n'est pas nécessaire) `https://cloud.influxdata.com/`. Lors de la création du compte, choisissez plutôt un fournisseur en Europe (par exemple, Azure Amsterdam). La rétention des données du compte gratuit est limitée à 30 jours.

La documentation d'InfluxDB cloud est disponible ici : `https://docs.influxdata.com/influxdb/v2.0/get-started/#start-with-influxdb-cloud-2-0`

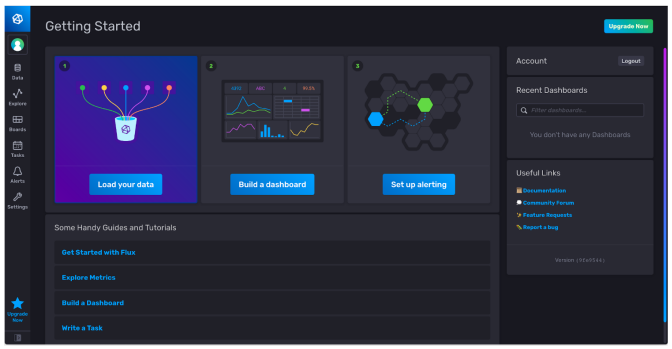


Figure 32: Interface Influx Data Cloud 1

1. Générez un token depuis le menu **Data ¿ onglet Tokens**
2. Ajoutez quelques mesures factices au bucket `lorawan` avec ces commandes shell (il faut configurer les variables `DBHOST` et `TOKEN`)

```
# DBHOST dépend du choix du provider Cloud (Azure) et du DataCenter (westeurope-1)
DBHOST=https://westeurope-1.azure.cloud2.influxdata.com
```

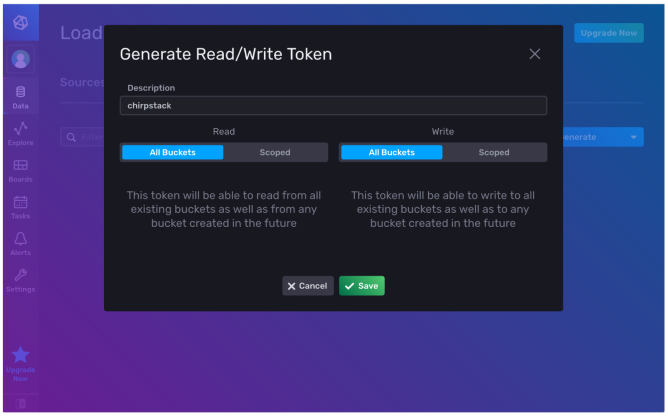


Figure 33: Interface Influx Data Cloud generation token

```
# L'adresse utilisée pour créer le compte InfluxData
DBUSERNAME='prenom.nom@gmail.com'
# Le token généré depuis la console InfluxData
BTOKEN='ov8MaASWoEXXXXXXXXXXXXXXMHTgmUvKjXXXXXXXXXXXXXXXXXXXXX-kHrD6p4MD-RoWhLc-2AXXX5G4_ajqQ=
curl -i -XPOST -u "$DBUSERNAME:$DBTOKEN" "$DBHOST/write?db=lorawan" \
--data-binary 'data,deveui=1234567890abdcef,application=WYRES_123 temperature_1=24,humidity_1=
sleep 10
curl -i -XPOST -u "$DBUSERNAME:$DBTOKEN" "$DBHOST/write?db=lorawan" \
--data-binary 'data,deveui=1234567890abdcef,application=WYRES_123 temperature_1=20,humidity_1=
sleep 10
curl -i -XPOST -u "$DBUSERNAME:$DBTOKEN" "$DBHOST/write?db=lorawan" \
--data-binary 'data,deveui=1234567890abdcef,application=WYRES_123 temperature_1=30,pressure_1=
sleep 10
curl -i -XPOST -u "$DBUSERNAME:$DBTOKEN" "$DBHOST/write?db=lorawan" \
--data-binary 'data,deveui=1234567890abdcef,application=WYRES_123 temperature_1=30,pressure_1=
sleep 10
curl -i -XPOST -u "$DBUSERNAME:$DBTOKEN" "$DBHOST/write?db=lorawan" \
--data-binary 'data,deveui=1234567890abdcef,application=WYRES_123 temperature_1=37,humidity_1=
```

1. Explorez les mesures enregistrées au moyen de l'explorateur de données. Cliquez sur **Submit** pour visualiser le graphe.

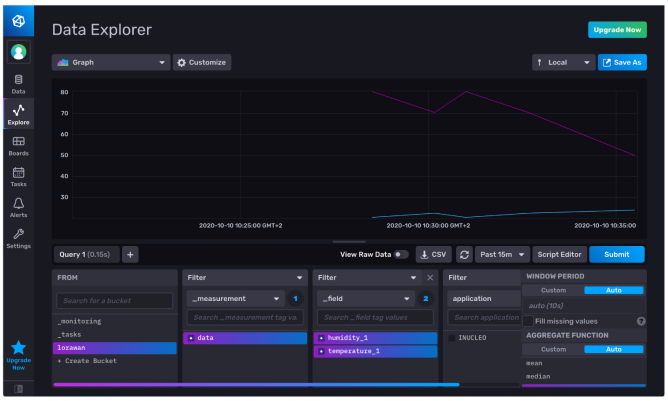


Figure 34: Interface InfluxData cloud Data Explorer

2. Créez un tableau de bord en ajoutant 2 cellules (cell). La cellule **Humidité** peut être rapidement obtenue en clonant la première cellule créée.



Figure 35: Interface Influx Data cloud Mon premier tableau de bord IoT

3. Créez une intégration de votre application `WYRES_123` depuis l'onglet **Integrations** de celle-ci. Utilisez les informations données (Token dans **Password**) dans la console InfluxData.

4. `https://westeurope-1.azure.cloud2.influxdata.com/write?db=lorawan`

Un nouveau bucket (`lorawan?db=lorawan`) est créé dans InfluxData. Attention, il faut prévoir de supprimer des buckets dans la console InfluxData car leur nombre est limité avec le compte gratuit. Vous pouvez changer la rétention du bucket (30 jours maximum avec les comptes gratuits). Vous pouvez explorer les données de vos endpoints et construire des tableaux de bord comme vu précédemment.

9.5 Autres plateformes IoT : Collecte, stockage et affichage des mesures de capteurs LoRaWAN

Nous présentons ci-dessous, les principales plateformes IoT :

9.5.1 MyDevices

- MyDevices est une plateforme IoT qui permet de connecter, déployer et gérer facilement des appareils connectés [14].
- Elle offre des outils de développement, de gestion et d'analyse des données IoT.
- MyDevices prend en charge une grande variété de protocoles et de normes IoT [14].

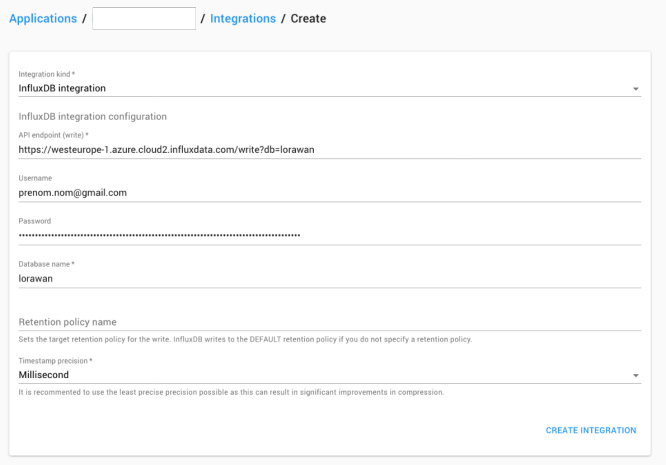


Figure 36: Interface serveur chipstark intergration influxDB

9.5.2 Datacake

- Datacake est une plateforme IoT complète pour la collecte, le traitement et la visualisation de données provenant d'appareils connectés.
- Elle permet de développer rapidement des applications IoT sans nécessiter de compétences techniques approfondies.
- Datacake propose des fonctionnalités avancées telles que l'analyse prédictive, l'automatisation et l'intégration à des services tiers.

9.5.3 Azure IoT

- Azure IoT est l'offre IoT de Microsoft, intégrée à la plateforme Azure [14].
- Elle comprend des services cloud pour connecter, surveiller et gérer des appareils IoT à grande échelle.
- Azure IoT propose des fonctionnalités étendues en matière de sécurité, d'analyse de données et d'intégration avec d'autres services Azure.

9.5.4 Amazon IoT

- Amazon IoT est la suite de services IoT d'Amazon Web Services (AWS) [14].
- Elle permet de connecter facilement des appareils IoT au cloud AWS et de développer des applications IoT.
- Amazon IoT offre des capacités avancées en matière de gestion des appareils, de traitement des données et d'analyse.

9.5.5 IBM IoT

- IBM IoT fait partie de la plateforme cloud IBM Watson, offrant des services IoT robustes.
- Elle permet de connecter, gérer et analyser des données provenant d'appareils IoT.
- IBM IoT intègre des fonctionnalités d'intelligence artificielle et d'apprentissage automatique pour enrichir les analyses.

Les plateformes IoT proposent des fonctionnalités complémentaires pour faciliter le déploiement et la gestion d'écosystèmes IoT à l'échelle de l'entreprise.

9.6 Collecte des mesures de capteurs LoRaWAN en ligne de commande:

NB : Attention, le port 8883 (MQTT/TLS) est filtré par Eduroam et Wifi Campus. [15]

9.6.1 Installation de nvm

Installez nvm (Node Version Manager) Installez jq pour reformater les messages MQTT.

```
nvm list
nvm install stable
nvm use stable
npm install -g mqtt
npm install -g json2csv
```

json2csv convertit un fichier JSON ou NDJSON en fichier CSV.

9.6.2 Collecte

Récupérez vos credentiels MQTT auprès de l'enseignant.

```
MQTT_BROKER=lns.campusiot.imag.fr
MQTT_PORT=8883
MQTT_USERNAME=org-XX
MQTT_PASSWORD=XXXXXXXXXXXXXXX
```

Vous pouvez souscrire au topic MQTT de votre application (251 dans l'exemple ci-dessous)

```
Pour souscrire au flux des endpoints de l'application 251

APP_ID=251
TOPIC=application/$APP_ID/device/+/rx

mqtt subscribe --insecure -v -h $MQTT_BROKER -p $MQTT_PORT -u $MQTT_USERNAME -P $MQTT_PASSWORD -l mqtt

mqtt subscribe --insecure -h $MQTT_BROKER -p $MQTT_PORT -u $MQTT_USERNAME -P $MQTT_PASSWORD -l mqtts $

mqtt subscribe --insecure -h $MQTT_BROKER -p $MQTT_PORT -u $MQTT_USERNAME -P $MQTT_PASSWORD -l mqtts $
jq -c '. | { deviceName:.deviceName, devEUI:.devEUI, time: .rxInfo[0].time, object:.object }'
```

9.6.3 Convertion NDJSON vers CSV

Petite Remarque

- Fichier NDJSON: Newline Delimited JSON : est un format fichier basé sur le Javascript, il se compose d'un enregistrement au format JSON par ligne.
- Fichier CSV: est un type de fichier qui stocke les valeurs en les séparant par des virgules.

```
mqtt subscribe --insecure -h $MQTT_BROKER -p $MQTT_PORT -u $MQTT_USERNAME -P $MQTT_PASSWORD -l mqtts $
jq -c '. | { deviceName:.deviceName, devEUI:.devEUI, time: .rxInfo[0].time, object:.object }' > /tmp/$
^C
json2csv -n --flatten-objects -i /tmp/$APP_ID.ndjson
```

Les fichiers sont facilement utilisables dans un notebook Jupyter.

9.7 Notebook Jupyter : Collecte, stockage et affichage des mesures de capteurs LoRaWAN :

Dans cette partie, vous apprendrez à analyser des mesures de capteurs LoRaWAN archivés au moyen d'un notebook Jupyter[16]. Vous trouvez sur ce dépôt CampusIoT/datasets un certain nombre des datasets contenant les mesures recueillies sur des capteurs Wyres Base installés à Grenoble, France. La collecte a duré plusieurs mois. Ces datasets peuvent servir à élaborer des notebooks Jupyter comme ces ébauches. Vous pouvez aussi analyser les données exportées (au format CSV) de la base de données InfluxDB comme montré au chapitre 8a.

9.7.1 Explication du code

- Importation des bibliothèques nécessaires : pandas pour la manipulation des données et matplotlib.pyplot pour la visualisation.
- Configuration de la connexion à la plateforme LoRaWAN : définition de la clé API et de l'endpoint de l'API.
- Collecte des données des capteurs : spécification de la période de temps et de la liste des capteurs. Utilisation de la méthode requests.get() pour récupérer les données via l'API LoRaWAN.
- Stockage des données dans un DataFrame Pandas : conversion des données au format Pandas et définition de l'index avec la date.
- Visualisation des données : création d'un graphique avec matplotlib.pyplot pour afficher les mesures des différents capteurs.

Ce notebook Jupyter [16] vous permet de collecter, stocker et visualiser facilement les données provenant de vos capteurs LoRaWAN. Vous pouvez ensuite étendre ce code pour ajouter des fonctionnalités supplémentaires, comme l'analyse des données ou l'intégration à d'autres services.

10 Langages de programmation de haut niveau avec RIOT

Dans ce chapitre, vous découvrirez comment programmer le firmware de votre carte Wyres Base en utilisant des langages de programmation autres que le C. Cependant, le degré de maturité des portages vers ces autres langages dépend de l'intégration des pilotes et des bibliothèques C de la plateforme RIOT [17].

Tout d'abord, il ne faut pas oublier de configurer la carte cible pour `make`

```
export BOARD=wyres-base
export EXTERNAL_BOARD_DIRS=~/github/campusiot/RIOT-wyres/boards
```

10.1 C++

Pour que le code C++ puisse être compilé avec RIOT, les étapes suivantes sont nécessaires :

- Tous les fichiers C++ doivent avoir l'extension de fichier `.cpp`[17], et tous les en-têtes C++ `.hpp` Pour le code externe, on peut utiliser la variable `SRCXXEXT` pour compiler les fichiers C++ avec d'autres extensions, comme `.cxx`
- `FEATURES_REQUIRED += cpp` doit être ajouté au Makefile de l'application Si la bibliothèque `libstdc++` est utilisée, `FEATURES_REQUIRED += libstdcpp` doit être ajouté en plus

Pour compiler et flasher le projet en utilisant 4 processeurs simultanément, entrez dans le dossier spécifique contenant des tests pour les constructeurs C++ et tapez la commande pour flasher[17].

```
cd ~/github/RIOT-OS/RIOT/tests/sys/cpp_ctors
make -j 4 flash
```

Si tout se passe bien, vous allez voir le resultat de la figure suivante dans votre console
Afin de compiler et flasher le projet en utilisant 4 processeurs simultanément, entrez dans le dossier spécifique contenant des tests relatifs aux threads C++11

```
cd ~/github/RIOT-OS/RIOT/tests/sys/cpp11_thread
```

Executez la commande suivante

```
make -j 4 flash
```

Après l'execution des commandes, vous devriez voir le resultat de la figure suivante

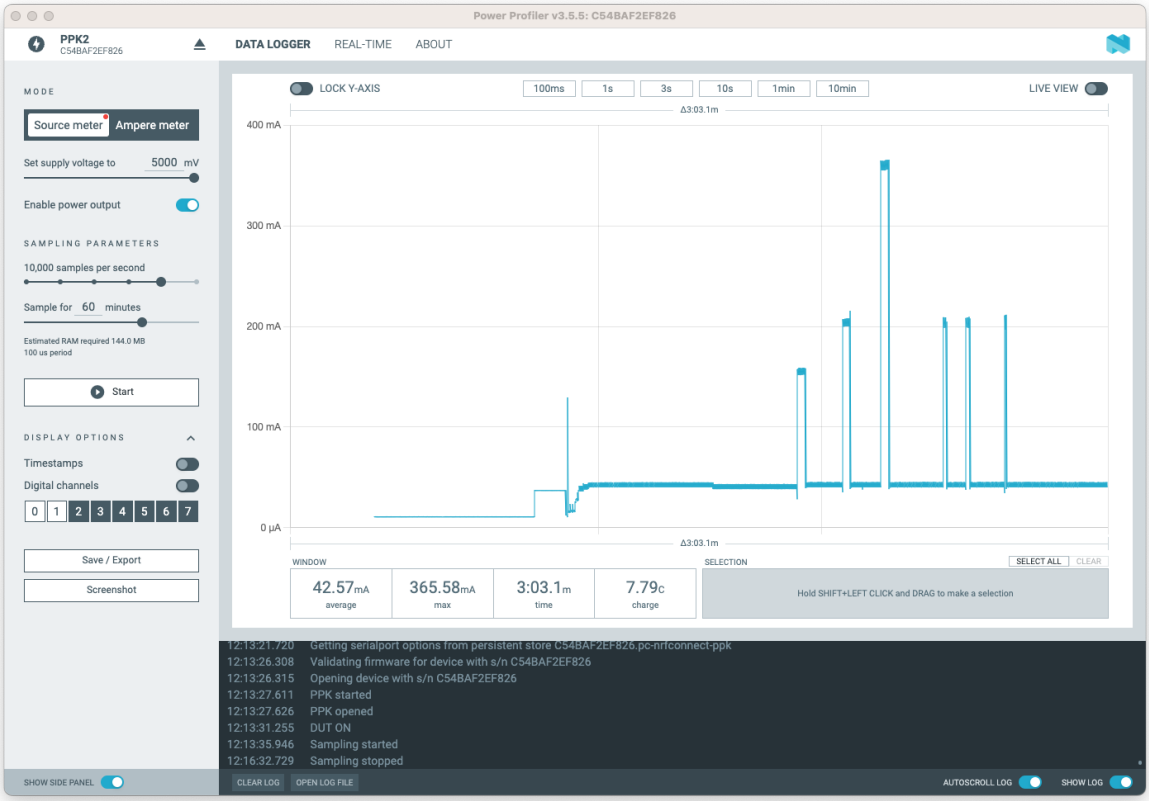


Figure 40: Informations sur la consommation d'énergie en temps réel

13.1.2 ST X-NUCLEO-LPM01A Power Shield

La carte X-NUCLEO-LPM01A permet de surveiller la consommation de cartes embarquées. Elle est faite pour alimenter et monitorer la consommation des cartes STM32 Nucleo 32,64 et 144. Elle est utilisable avec toutes les cartes des autres constructeurs (avec pinout Arduino ou Arduino Nano ou autre).

Elle est utilisable avec l'environnement de ST (via un port série) ou en standalone.

ST fournit le firmware du benchmark ULPBench pour ses cartes.



Figure 41: Le X-NUCLEO-LPM01A

Visitez le site `https://github.com/CampusIoT/tutorial/tree/master/x-nucleo-lpm01a` pour en savoir plus sur le X-NUCLEO-LPM01A

13.2 Principes de programmation basse consommation d'énergie

13.2.1 Gestion de l'énergie dans RIOT

Les mécanismes de gestion d'énergie dans RIOT s'appuient sur la capacité des microcontrôleurs de basculer dans des modes de fonctionnement à très basse consommation.
Dans RIOT, le principe est de basculer automatiquement le microcontrôleur dans l'un de ces modes lorsque toutes les tâches sont terminées ou bloquées en attente d'un évènement extérieur.

Il faut configurer la carte cible pour make

```
 export BOARD=wyres-base
export EXTERNAL_BOARD_DIRS=~/github/campusiot/RIOT-wyres/boards
```

Ensuite, changer le répertoire de travail vers le répertoire tests/periph/pm du projet RIOT-OS. A la fin, il faut compiler le test periph/pm et le flasher sur la carte wyres-base définie précédemment.

```
cd ~/github/RIOT-OS/RIOT/tests/periph/pm
gmake flash
```

Le message : "The available power modes are 0 - 2. Lower-numbered power modes save more power, but may require an event/interrupt to wake up the CPU. Reset the CPU if needed." explique que les modes d'alimentation disponibles sont 0, 1 et 2, et que les modes les plus bas (0 et 1) consomment moins d'énergie mais nécessitent

```
> main(): This is RIOT! (Version: 2023.07-devel-325-g2863d)
This application allows you to test the CPU power management.
The available power modes are 0 - 2. Lower-numbered power modes
save more power, but may require an event/interrupt to wake up
the CPU. Reset the CPU if needed.
mode 0 blockers: 1
mode 1 blockers: 2
mode 2 blockers: 0
Lowest allowed mode: 2
using BTN0 as wake-up source
> reboot
main(): This is RIOT! (Version: 2023.07-devel-325-g2863d)
This application allows you to test the CPU power management.
The available power modes are 0 - 2. Lower-numbered power modes
save more power, but may require an event/interrupt to wake up
the CPU. Reset the CPU if needed.
mode 0 blockers: 1
mode 1 blockers: 2
mode 2 blockers: 0
Lowest allowed mode: 2
using BTN0 as wake-up source
```

Figure 42: Programme pour tester les différents modes d'alimentation

```
> set_rtc 0 5
Setting power mode 0 for 5 seconds.
main(): This is RIOT! (Version: 2023.07-devel-325-g2863d)
This application allows you to test the CPU power management.
The available power modes are 0 - 2. Lower-numbered power modes
save more power, but may require an event/interrupt to wake up
the CPU. Reset the CPU if needed.
mode 0 blockers: 1
mode 1 blockers: 2
mode 2 blockers: 0
Lowest allowed mode: 2
> set_rtc 1 5
Setting power mode 1 for 5 seconds.
> set_rtc 2 5
Setting power mode 2 for 5 seconds.
The alarm rang
```

Figure 43: La suite du programme

un événement ou une interruption pour réveiller le processeur. Il est également mentionné qu'un redémarrage du processeur peut être nécessaire.

- "mode 0 blockers: 1"

- "mode 1 blockers: 2"
- "mode 2 blockers: 0"
- "Lowest allowed mode: 2"

Ces lignes indiquent le nombre de "blocages" (éléments empêchant le passage à un mode d'alimentation plus faible) pour chaque mode d'alimentation, ainsi que le mode le plus faible autorisé, qui est ici le mode 2.

Le message : "using BTN0 as wake-up source" indique que le bouton BTN0 est utilisé comme source de réveil du processeur.

Les commandes : `set_rtc 0 5`, `set_rtc 1 5`, `set_rtc 2 5` permettent de définir le mode d'alimentation (0, 1 ou 2) pendant 5 secondes.

Le message : "The alarm rang" indique que l'alarme (qui était réglée lors de la dernière commande `set_rtc 2 5`) a sonné, signalant la fin de la période de 5 secondes.

La figure suivante présente une observation de l'impact des modes d'alimentation en termes de consommation d'énergie

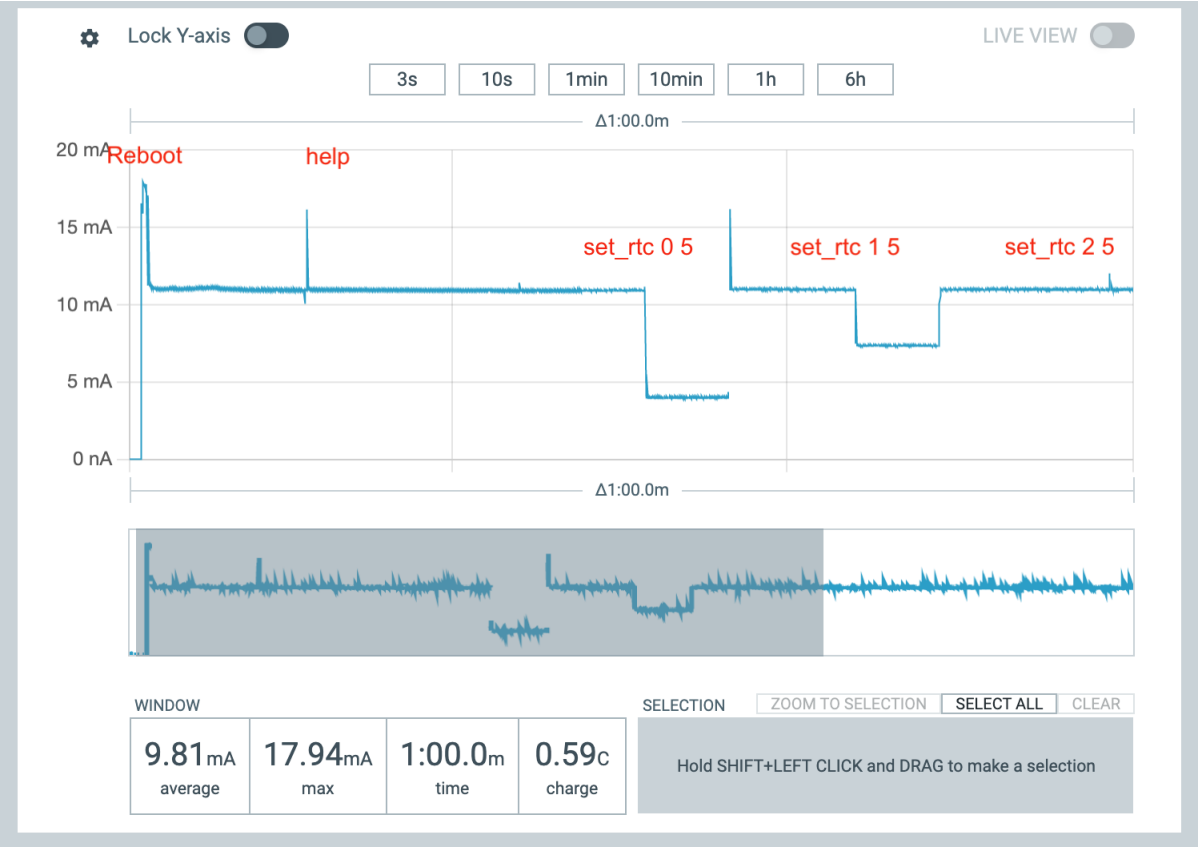


Figure 44: Observation de l'impact des modes d'alimentation en termes de consommation d'énergie.

La figure suivante explique que le système est passé en mode d'alimentation 0, le plus économe en énergie, et qu'il attend maintenant un événement de réveil. Il rappelle ensuite les informations sur les différents modes d'alimentation disponibles, à savoir que les modes les plus bas (0 et 1) consomment moins mais peuvent nécessiter un événement pour réveiller le CPU, et que le mode le plus bas autorisé est ici le mode 2.

```
> pm set 0
CPU is entering power mode 0.
Now waiting for a wakeup event...
main(): This is RIOT! (Version: 2023.07-devel-325-g2863d)
This application allows you to test the CPU power management.
The available power modes are 0 - 2. Lower-numbered power modes
save more power, but may require an event/interrupt to wake up
the CPU. Reset the CPU if needed.
mode 0 blockers: 1
mode 1 blockers: 2
mode 2 blockers: 0
Lowest allowed mode: 2
```

Figure 45: Le mode d'alimentation 0

CONCLUSION

En conclusion, ce tutoriel se concentre sur l'exploration approfondie de RIOT OS et de la carte Wyres Base dans le contexte de l'IoT. En exploitant les capacités de LoRa et LoRaWAN, nous avons exploré le pilotage des capteurs, la transmission de données à longue portée et la gestion efficace de l'énergie. Nous avons débuté par les bases des entrées-sorties et des capteurs MEMS, puis approfondi les communications avancées avec LoRa et LoRaWAN. Des sujets avancés tels que la cryptographie et l'intelligence artificielle embarquée ont également été abordés, tout en envisageant des contributions potentielles à l'écosystème RIOT OS. L'objectif principal de ce tutoriel était d'inspirer et d'équiper les développeurs pour créer des solutions IoT innovantes et écoénergétiques avec la carte Wyres Base et RIOT OS. Au terme de notre tutoriel nous pouvons dire que ce challenge a été relevé avec sucés et chaque développeurs qui utilisera ce document sera capable de fournir un résultat pertinent à la fin de l'utilisation de ce tutoriel.

REFERENCES BIBLIOGRAPHIQUES

[1] Didier Donsez,RIOT online course, [en ligne], https://github.com/RIOT-OS/riot-course. Consulté le : 10/07/2024

[2] Didier Donsez, Installation de RIOT, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/01.md . Consulté le : 08/07/2024

[3]Didier Donsez, La carte Wyres Base, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/02.md . Consulté le : 09/07/2024

[4]Didier Donsez, La carte Wyres Base, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/03.md . Consulté le 10/07/2024

[5]Didier Donsez, La carte Wyres Base, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/04.md . Consulté le 11/07/2024

[6]Didier Donsez, La carte Wyres Base, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/05.md . Consulté le 10/07/2024

[7]Didier Donsez, La carte Wyres Base, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/06.md . Consulté le 10/07/2024

[8]Didier Donsez, , [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/07.md . Consulté le 10/07/2024

[9]Didier Donsez, Collecte, stockage et affichage des mesures de capteurs LoRaWAN, [en ligne], https://github.com/CampusIoT/Rwyres/blob/main/tutoriel/08.md . Consulté le 10/07/2024

[10]Didier Donsez, Nodered - Influxdb - Grafana (on premise) , [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/08a.md . Consulté le 10/07/2024

[11]Didier Donsez, Thingsboard (cloud) : Collecte, stockage et affichage des mesures de capteurs LoRaWAN, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/08b.md . Consulté le 10/07/2024

[12]Didier Donsez, Thingsboard (on premise) :: Collecte, stockage et affichage des mesures de capteurs LoRaWAN, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/08c.md . Consulté le 10/07/2024

[13]Didier Donsez, Influx Data (cloud) :: Collecte, stockage et affichage des mesures de capteurs LoRaWAN, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/08d.md . Consulté le 10/07/2024

[14]Didier Donsez,Autres plateformes IoT :: Collecte, stockage et affichage des mesures de capteurs LoRaWAN, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/08e.md . Consulté le 10/07/2024

[15]Didier Donsez,Collecte des mesures de capteurs LoRaWAN en ligne de commande, [en ligne], https://github.com/CampusIoT/wyres/blob/main/tutoriel/08f.md . Consulté le 10/07/2024

[16]Didier Donsez,Notebook Jupyter :: Collecte, stockage et affichage des mesures de capteurs LoRaWAN, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/08g.md . Consulté le 10/07/2024

[17]Didier Donsez,Langages de Programmation de haut niveau avec RIOT, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/09.md . Consulté le 10/07/2024

[18]Didier Donsez,Cryptographie avec RIOT et la carte Wyres Base, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/10.md . Consulté le 13/07/2024

[19]Didier Donsez,Cryptographie avec RIOT et la carte Wyres Base, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/10.md . Consulté le 13/07/2024

[20]Didier Donsez,Intelligence artificielle embarquée (TinyML), [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/11.md . Consulté le 14/07/2024

[21]Didier Donsez,Gestion d'énergie avec RIOT et la carte Wyres Base, [en ligne], https://github.com/CampusIoT/RIOT-wyres/blob/main/tutoriel/12.md . Consulté le 14/07/2024